SĂPTĂMÂNA 9

Modelarea bazată pe agenți și sisteme complexe

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Automate celulare: reguli **locale**, comportament **global** – Game of Life, trafic, percolare (S8)
- ✓ Particule: Newton, Verlet, grile și arbori pentru vecini; performanță și paralelism
- ✓ SIR și SEIR (S7): populații **omogen amestecate**, toți indivizii identici
- ✓ Întrebarea de final din S8: cum ajunge segregat un oraș în care **nimeni** nu vrea segregare?

La finalul acestei săptămâni veți putea…

1. să definiți un **agent** (percepție, stare, comportament, acțiune) și să distingeți agenții reactivi de cei cu stare și cei care învață
2. să alegeți **mediul** și **topologia** interacțiunilor (grilă, spațiu continuu, rețea) și modul de actualizare (sincron, asincron)
3. să implementați modele clasice: **Boids**, **Schelling**, furnicile lui Deneubourg, chemotaxia, El Farol
4. să comparați un model **SIR cu agenți** cu modelul cu ecuații și să explicați rolul rețelelor (lume mică, fără scară)
5. să descrieți un model cu protocolul **ODD** și să îl calibrați și validați (replicări, ABC, tipare)
6. să discutați critic **agenții generativi** bazați pe modele de limbaj și problema validării lor

Ecuații pentru populații sau reguli pentru indivizi?

Modele cu ecuații (S7)

- variabile **agregate**: S(t), I(t), R(t)
- toți indivizii sunt **identici**
- oricine se poate întâlni cu oricine
- parametrii sunt medii: β, γ
- rapide, analizabile, ușor de calibrat

vs

Modele bazate pe agenți (S9)

- fiecare **individ** este simulat
- atribute diferite: vârstă, comportament, poziție
- interacțiuni **locale**, în spațiu sau pe o rețea
- reguli de decizie, memorie, învățare
- comportamentul global **emerge** din interacțiuni

Holban (cap. 7): marea deficiență a modelelor cu ecuații este că descriu doar parțial complexitatea sistemului real, în primul rând prin absența mediului geografic. Adăugarea mediului și a relațiilor dintre entități a condus la modelarea bazată pe agenți.

Presupunerile ascunse ale modelului Kermack–McKendrick

- Mărimea populației este **fixă**, fără nașteri; decesele apar doar din boală sau din cauze naturale
- Incubația este **instantanee**, iar durata bolii este **aceeași** pentru orice persoană
- Populația este **omogenă**: nu contează vârsta, structura socială, localizarea
- Contactele sunt **aleatoare**: un bolnav din Timișoara poate infecta pe oricine din țară
- Coeficienții sunt constanți – în realitate oamenii își schimbă comportamentul în timpul epidemiei

H **Model comportamental**

Holban: un model în care coeficienții ecuațiilor variază în timp. Când variațiile vin din deciziile indivizilor, e mai natural să modelăm direct indivizii.

Unde se folosesc modelele bazate pe agenți

1 Epidemii

Răspândirea bolilor în orașe reale, cu școli, locuri de muncă și transport; efectul măsurilor țintite.

2 Trafic și evacuări

Șoferi și pietoni cu rute și decizii proprii: ambuteiaje, stadioane, clădiri în caz de incendiu.

3 Economie și piețe

Comercianți cu strategii diferite: bule, prăbușiri, formarea prețurilor (Cernazean: bursa din New York).

4 Ecologie

Animale care caută hrană, prădători și pradă în spațiu, stoluri, migrații.

5 Societate

Segregare rezidențială, răspândirea opiniilor și a zvonurilor, cooperare, norme sociale.

6 Informatică

Roboți în roi, rețele de senzori, personaje în jocuri, testarea sistemelor distribuite.

Ora 1

De la ecuații la agenți

1. Agentul: percepție, stare, comportament, acțiune
2. Sisteme multi-agent: mediu, topologie, actualizare
3. Exemplul Holban: epidemia SEIRS cu 10 000 de agenți
4. Agenți care învață: barul El Farol

Ce este un agent

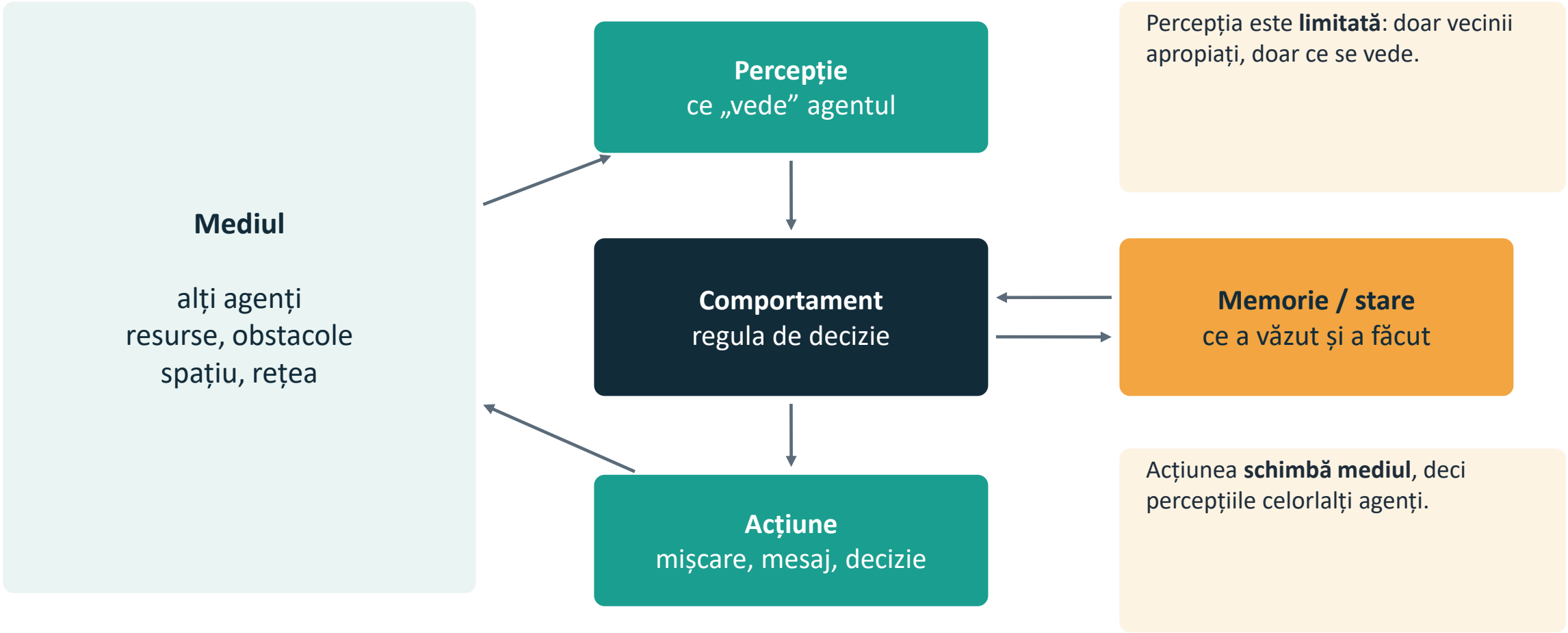
O entitate autonomă care percepe mediul și acționează asupra lui

Holban: un agent este o entitate care funcționează **continuu** și **autonom** într-un mediu în care au loc alte acțiuni și în care sunt prezenți și alți agenți. Conceptul vine din inteligența artificială (Cernazean L8).

Holban, cap. 7.2; Shoham și Leyton-Brown, Multiagent Systems, 2008

- **Poziționare**: primește informații din mediu și execută acțiuni care îl modifică
- **Autonomie**: își controlează singur acțiunile și starea internă, fără intervenție externă
- **Flexibilitate**: reacționează la timp, urmărește scopuri și interacționează cu alți agenți
- În ABM: atribute + reguli de interacțiune + un context (spațiu real sau virtual)

Agentul și mediul: bucla percepție – decizie – acțiune



Modelul abstract al unui agent (Holban)

$$acțiune : S^* \rightarrow A$$

$S = \{s_1, s_2, \ldots\}$	mulțimea stărilor posibile ale **mediului**; la orice moment mediul este într-una dintre ele
$A = \{a_1, a_2, \ldots\}$	mulțimea **acțiunilor** pe care agentul le poate executa
S^*	secvențe de stări: agentul decide pe baza **istoricului** h = s_0, a_0, s_1, a_1, …, nu doar a stării curente

Comportamentul este determinist dacă din aceeași situație agentul ajunge mereu în aceeași stare a mediului și nedeterminist (probabilist) altfel. Funcția de decizie se separă în două subsisteme: percepție : S → P și acțiune : P* → A.

Agent reactiv sau agent cu stare?

Reactiv (reflex simplu)

- acțiune : $S \to A$
- decide doar după **prezent**
- Cernazean: PERCEPȚIE → ACȚIUNE
- exemplu: termostatul simplu
- Holban: Game of Life, Boids

vs

Cu stare (cognitiv)

- acțiune : $I \to A$, următor : $I \times P \to I$
- are o **memorie** internă I
- Cernazean: PERCEPȚIE × STARE → ACȚIUNE × STARE
- poate **învăța** din experiență
- Holban: persoana din modelul SEIRS

Ciclul unui agent cu stare (Holban): pornește din starea internă i_o, observă mediul, generează percepția, își actualizează starea cu funcția „următor”, apoi execută acțiunea corespunzătoare noii stări – și reia ciclul.

Termostatul: reactiv vs cu stare (histerezis)

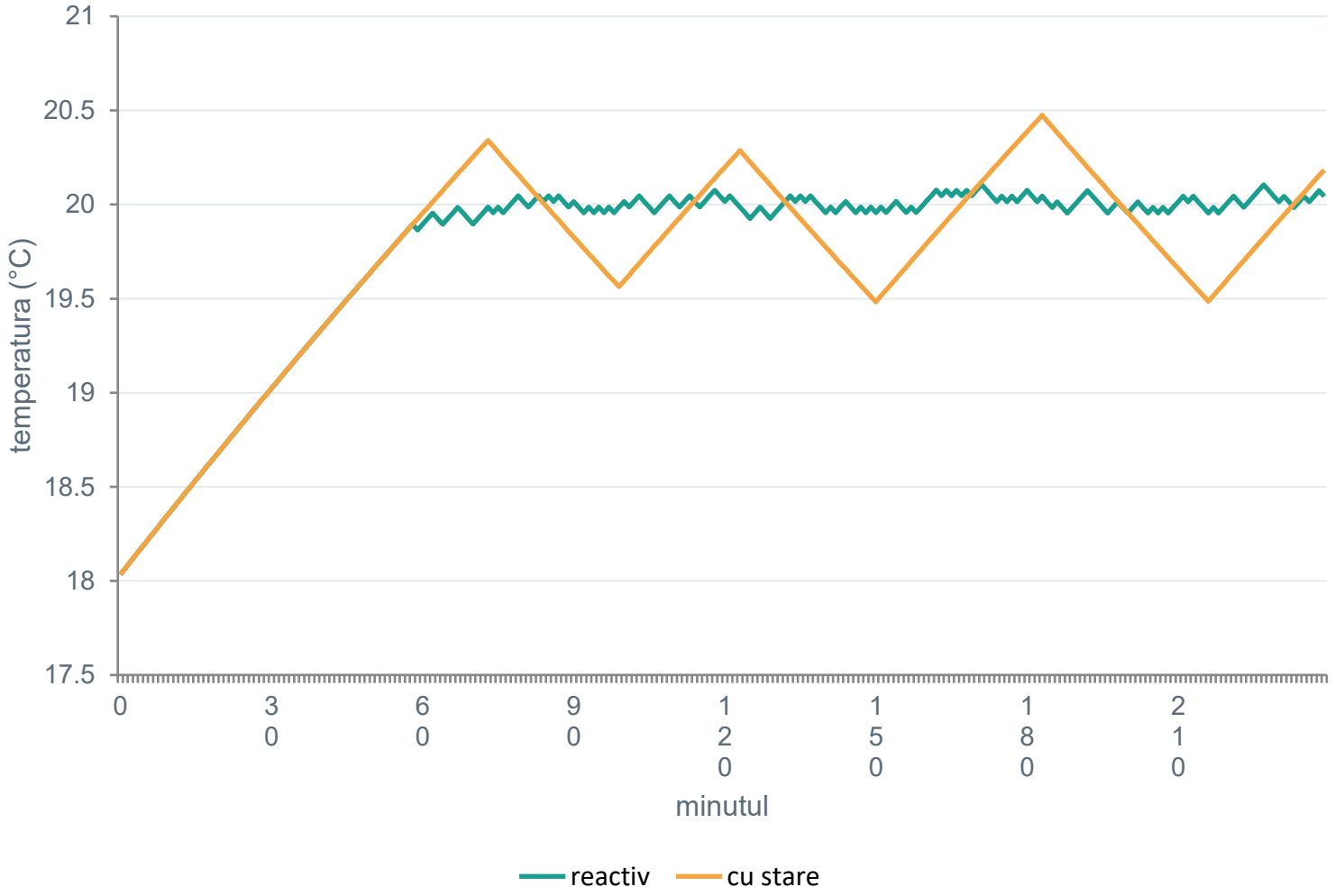
```
def reactiv(T_masurat, pornit):
    return T_masurat < 20.0              # doar prezentul

def cu_stare(T_masurat, pornit):
    if pornit:                           # memoria: ce făceam
        return T_masurat < 20.5
    return T_masurat < 19.5

T, pornit = 18.0, False
for minut in range(1440):              # o zi
    m = T + rng.normal(0, 0.1)           # senzor cu zgomot
    nou = agent(m, pornit)
    comutari += nou != pornit; pornit = nou
    T += -0.002 * (T - 5) + (0.06 if pornit else 0)
```

- Același mediu: cameră, exterior 5 °C, senzor cu zgomot de 0,1 °C
- Agentul cu stare își **amintește** dacă încălzirea e pornită
- Pragurile diferă după stare: 19,5 °C pentru pornire, 20,5 °C pentru oprire
- Cost: o singură variabilă de memorie

Primele 4 ore: aceeași temperatură medie, comportament foarte diferit



- Reactiv: **764** comutări pe zi – releul se uzează rapid
- Cu stare: **47** comutări pe zi
- Temperatura: 19,9–20,1 °C vs 19,4–20,5 °C
- Încălzirea funcționează 52% din timp în ambele cazuri

Arhitecturi de agenți, de la simplu la complex

Arhitectura	Decide pe baza	Exemplu în simulare
Reflex simplu	percepției curente	celula din Game of Life, boid, termostat simplu
Cu stare (model)	percepției și memoriei	persoana SEIRS, furnica cu memorie, bacteria
Orientat pe scop	unei ținte de atins	pieton care caută ieșirea, vehicul cu destinație
Orientat pe utilitate	unei funcții de câștig	comerciant, consumator, gospodărie
Care învață	experienței acumulate	jucătorii din El Farol, agenți cu învățare prin întărire

- Clasificare după Russell și Norvig; Holban folosește trei clase: total reactivi, de percepție, cu stări
- Regula de aur: agentul cel mai **simplu** care reproduce fenomenul – complexitatea vine din interacțiuni

Exemplu: un agent comerciant (Cernazean L8)

```
def comportament(pret, stare):
    ultimul_pret, bani, actiuni = stare
    if pret - ultimul_pret > RL:      # a crescut mult
        n = floor(actiuni * Cs)
        return VINDE(n, pret)
    elif ultimul_pret - pret > RL:    # a scăzut mult
        n = floor(bani * Cb / pret)
        return CUMPARA(n, pret)
    else:
        return NIMIC
```

- Starea: ultimul preț, banii, acțiunile deținute
- Parametri individuali: pragul RL, fracțiunile Cs și Cb
- Mii de comercianți cu parametri **diferiți** formează o piață
- Întrebarea lui Cernazean: putem explica evoluția pieței cunoscând strategiile individuale?

Mulți agenți, un mediu comun

1. Mediul și topologia interacțiunilor
2. Percepție limitată, interacțiuni directe și indirecte
3. Actualizare sincronă sau asincronă
4. Timpul: pași fixi sau evenimente

Sisteme multi-agent și modele bazate pe agenți

- Un **sistem multi-agent** (MAS): mai mulți agenți, identici sau diferiți, care interacționează direct sau prin mediu
- MAS nu înseamnă neapărat ABM: există MAS pentru optimizare, securitatea rețelelor, jocuri video (Cernazean)
- Un **model bazat pe agenți** (ABM) folosește un MAS pentru a modela un fenomen natural sau social
- Agenții pot avea o poziție în spațiu (2D, 3D, graf) și se pot **deplasa**
- Poziția influențează ce percep și cu cine interacționează

E **Ideea centrală**

Cernazean: modelăm entitățile de bază ca indivizi și observăm comportamentul global **emergent** – un comportament colectiv complex apare din comportamentele simple ale agenților.

Mediul: unde trăiesc agenții

1 Fără spațiu

Oricine poate întâlni pe oricine: echivalentul „omogen amestecat” al ecuațiilor.

2 Grilă

Celule discrete: Schelling, furnicile, Game of Life. Vecini ușor de găsit.

3 Spațiu continuu

Coordonate reale: Boids, bacterii, pietoni. Vecinii se caută cu grile sau arbori k-d (S8).

4 Rețea

Nodurile sunt agenți, muchiile sunt relații: prieteni, colegi, contacte, linkuri.

5 Hartă (GIS)

Străzi, clădiri, relief reale: evacuări, trafic, epidemii în orașe concrete.

6 Combinat

Mai multe straturi: locuință pe hartă, colegi într-o rețea, știri globale.

Cum interacționează agenții

1 Direct

Mesaje între agenți: contact de infectare (Holban), ofertă de cumpărare, invitație.

2 Prin mediu (stigmergie)

Agentul modifică mediul, alții reacționează: furnicile și cadavrele, feromonii, potecile.

3 Prin mărimi globale

Toți văd același semnal: prețul pieței, numărul de cazuri anunțat, aglomerația barului.

Topologia interacțiunilor (Cernazean): cine poate interacționa cu cine depinde de rază, de pereți și de unghiul de vedere – sau, într-o rețea, de muchii. Pentru N agenți, căutarea naivă a vecinilor costă $O(N^2)$; o grilă sau un arbore k-d o reduce, ca la particule (S8).

Percepția limitată: ce vede un agent

- **Rază**: agentul vede doar vecinii aflați la o distanță mai mică decât R (Boids, persoanele din modelul Holban: 1 km)
- **Unghi de vedere**: o pasăre nu vede în spate; un pieton privește înainte
- **Obstacole**: pereții blochează vederea și mișcarea (Cernazean: „environment awareness”)
- **Zgomot**: percepția poate fi imprecisă – senzorul termostatului, informații întârziate
- Cât percepe agentul este o alegere de modelare care schimbă rezultatul

G **Graurii**

Ballerini et al. (2008): fiecare graur dintr-un stol urmărește aproximativ 6–7 vecini, indiferent de distanță – interacțiune **topologică**, nu metrică.

Implementare: abordarea lagrangiană și cea eulerianăă (Cernazean)

Lagrangiană: lista de agenți

- fiecare agent își știe **poziția** exactă
- interacțiunile se calculează global
- naiv: $O(N^2)$ perechi
- cu arbore k-d: construcție O(N log N)
- precizie spațială completă

Eulerianăă: grila de celule

- mediul este o grilă regulată
- fiecare celulă are **lista agenților** din ea
- vecinii: celulele alăturate
- interacțiuni locale, ușor de paralelizat
- dezavantaj: pierdem din precizia spațială

În programarea orientată pe obiecte: clasa = tipul de agent, instanța = un agent, membrii privați = starea internă, metodele publice = comportamentul. Bibliotecile (Mesa, NetLogo) oferă ambele variante de spațiu.

Actualizare asincronă și sincronă (Cernazean L8)

```
# asincron: agenții acționează pe rând
while t < t_max:
    for agent in ordine_aleatoare(agenti):
        p = percepe(agent, mediu, agenti)
        actiune = agent.comportament(p)
        actualizeaza(mediu, actiune)   # imediat
    t += 1

# sincron: toți percep aceeași stare
while t < t_max:
    ps = [percepe(a, mediu, agenti) for a in agenti]
    acts = [a.comportament(p) for a, p in zip(agenti, ps)]
    actualizeaza_tot(mediu, acts)       # la final
    t += 1
```

- **Asincron**: fiecare agent vede efectele celor dinaintea lui
- Ordinea trebuie **amestecată** la fiecare pas, altfel primul agent are mereu avantaj
- **Sincron**: ca la automatele celulare; cere rezolvarea **conflictelor** (doi agenți vor aceeași celulă)
- Mesa 3: model.agents.shuffle_do(„step”)

Contează schema de actualizare? Regula majorității pe un inel

Sincron

- 100 de agenți pe un inel, opinii 0/1 alternante
- fiecare adoptă majoritatea dintre el și cei doi vecini
- toți își schimbă opinia **simultan**
- rezultat: **oscilează** la nesfârșit (perioada 2)

vs

Asincron

- aceeași stare inițială, aceeași regulă
- agenții se actualizează pe rând, în ordine aleatoare
- fiecare vede deja opiniile noi ale vecinilor
- rezultat: **punct fix** după 1 rundă: blocuri stabile de opinii

Oscilația sincronă este un **artefact** al schemei de actualizare, nu o proprietate a fenomenului: oamenii reali nu își schimbă opinia exact în aceeași clipă. Cernazean (L4): actualizarea asincronă evită oscilațiile, dar pierde paralelismul.

Timpul într-un model bazat pe agenți

- **Pas fix**: la fiecare pas, fiecare agent acționează o dată (Schelling, Boids, majoritatea modelelor)
- Pasul trebuie să corespundă unei durate reale: o zi în modelul Holban, o secundă în trafic
- **Evenimente discrete**: agenții programează evenimente (sfârșitul incubației, sosirea la serviciu) – legătura cu S5
- Cernazean: comportament(eveniment, stare) întoarce o stare nouă și o listă de evenimente noi
- Variante mixte: AnyLogic combină agenți, evenimente discrete și dinamica sistemelor

T **Alegerea pasului**

Un pas prea mare face ca evenimente care ar trebui să fie succesive să pară simultane; un pas prea mic irosește calcul. Ca la integrarea numerică (S7).

Verificare

? Într-un model cu actualizare **asincronă**, de ce amestecăm ordinea agenților la fiecare pas?

A ca simularea să ruleze mai repede

B ca niciun agent să nu aibă sistematic avantajul de a acționa primul

C pentru a putea paraleliza calculul

D pentru că altfel modelul devine sincron

Exemplul din curs (Holban, cap. 7)

O epidemie SEIRS cu 10 000 de agenți

1. Ipotezele modelului
2. Agentul ca diagramă de stări
3. Spațiu vs amestec omogen vs ecuații
4. Două neconcordanțe în textul original

Ipotezele modelului Holban

Element	Valoare
Populația	10 000 de persoane, distribuite uniform pe o suprafață de 10 × 10 km
Vecinii	fiecare persoană cunoaște doar oamenii aflați la cel mult **1 km**
Start	10 persoane infectate aleator; restul susceptibile, nimeni imun
Contacte	un infectat contactează cel mult **5** vecini pe zi; transmisie cu probabilitatea **0,1**
Latența (Expus)	uniform 3–6 zile; persoana nu este încă contagioasă
Boala (Infecțios)	uniform 7–15 zile
Imunitatea	uniform 30–70 de zile, apoi persoana redevine susceptibilă

Agentul ca diagramă de stări (SEIRS)

1

Susceptibil

primește un mesaj „infecție” de la un vecin bolnav: trece în Expus cu probabilitatea 0,1

2

Expus

latență de 3–6 zile (calculată la intrare); nu infectează pe nimeni

3

Infecțios

7–15 zile; în fiecare zi trimite mesaje la 5 vecini aleși la întâmplare

4

Recuperat

imun 30–70 de zile, apoi revine la Susceptibil

ciclul se închide: Recuperat → Susceptibil (imunitatea expiră)

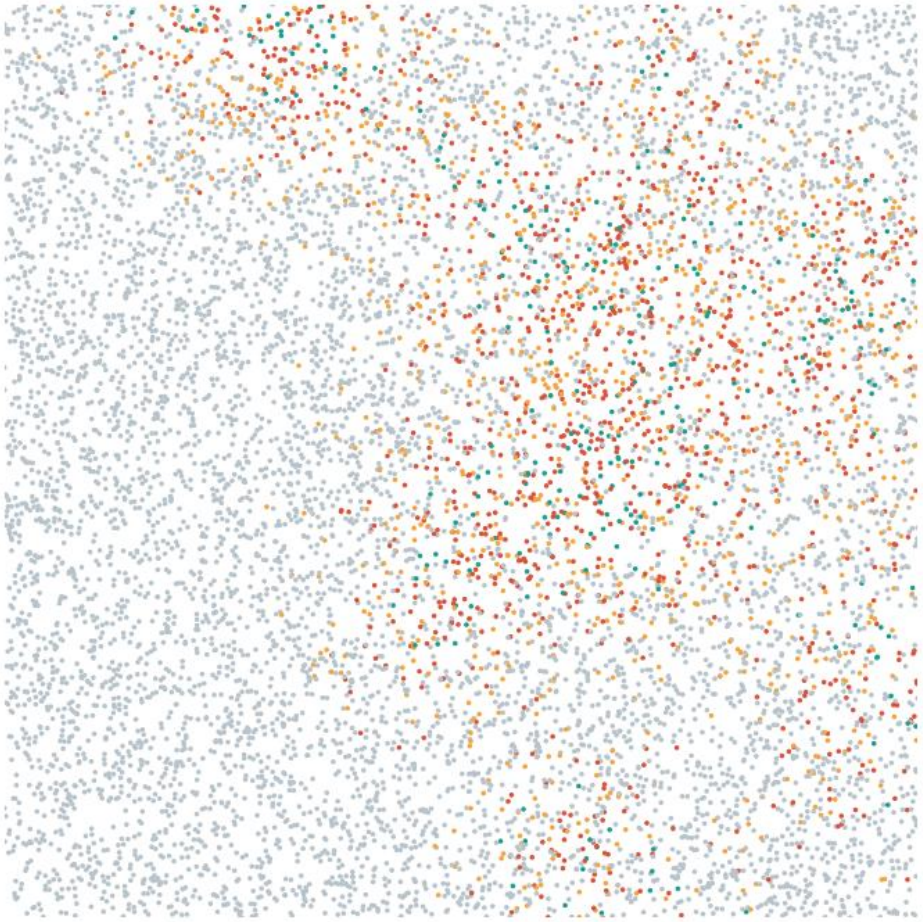
O zi din simulare, vectorizată cu NumPy

```
inf = np.flatnonzero(st == I)
src = np.repeat(inf, 5)                  # 5 contacte / infectat
k = (rng.random(len(src)) * grad[src]).astype(int)
tgt = vecini[start[src] + k]           # vecini aleatori
noi = tgt[(st[tgt] == S) & (rng.random(len(tgt)) < 0.1)]

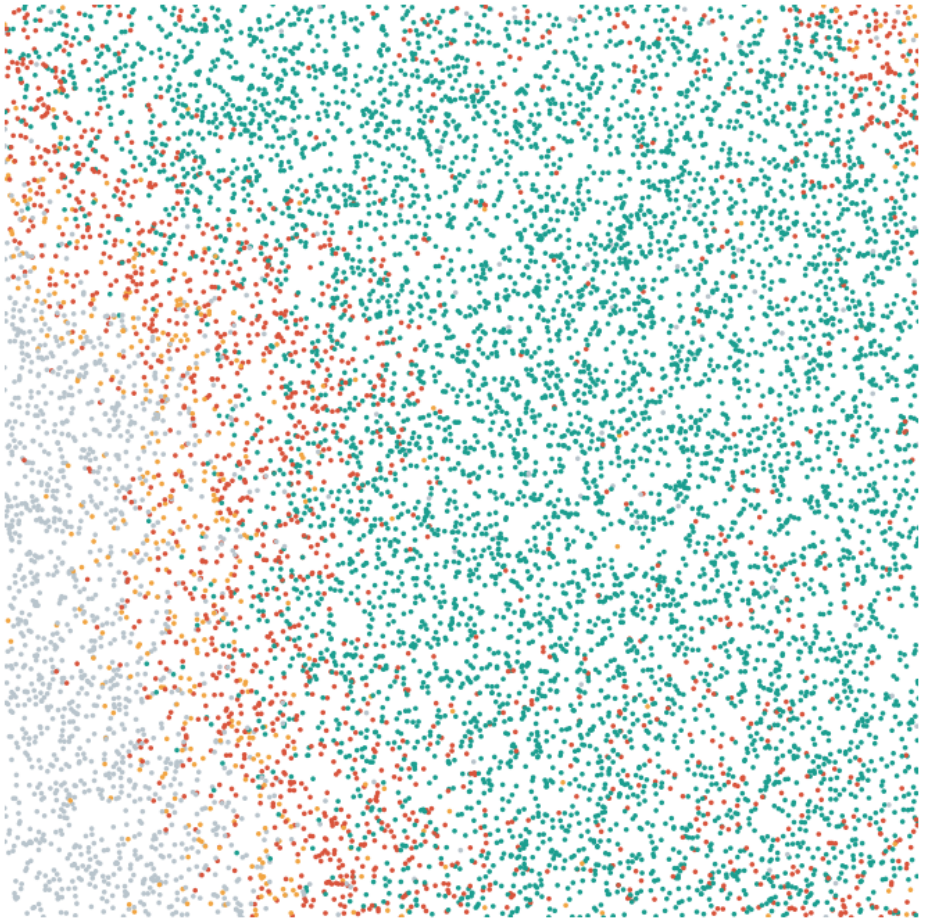
timer -= 1
st[(st == E) & (timer <= 0)] = I       # (plus durate noi)
st[(st == I) & (timer <= 0)] = R
st[(st == R) & (timer <= 0)] = S       # imunitatea expiră
st[noi] = E; timer[noi] = rng.integers(3, 7, len(noi))
```

- Vecinii pe 1 km: găsiți o singură dată cu cKDTree.query_ball_point (S8)
- ~314 de vecini în medie pentru fiecare persoană
- 365 de zile pentru 10 000 de agenți: ~0,6 s
- Varianta „fără spațiu”: vecinii aleși din toată populația

Unda epidemică traversează orașul

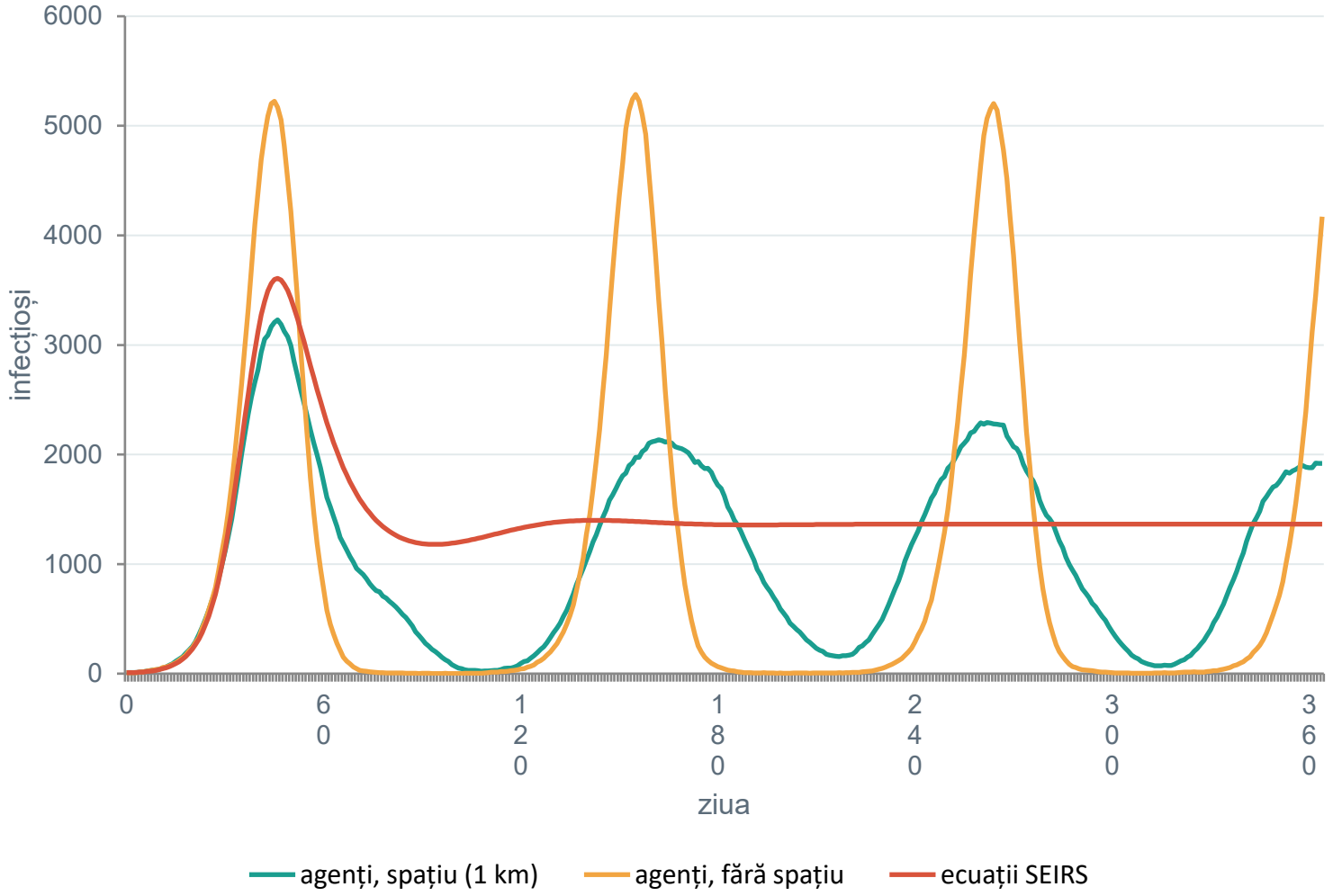


ziua 30: 1 121 infecțioși (roșu), expuși (galben), recuperați (verde)



ziua 60: unda a trecut; 1 742 infecțioși, la marginea orașului

Infecțioși în timp: spațiu, amestec omogen, ecuații



- Aceeași medie a parametrilor: R_0 = 5 × 0,1 × 11 = **5,5**
- Primul vârf: 3 229 (spațiu), 5 224 (fără spațiu), 3 608 (ecuații)
- Ecuațiile se liniștesc într-un echilibru; agenții produc **valuri** repetate
- Între valuri, fără spațiu rămân doar 1–2 infecțioși

Ce schimbă spațiul și indivizii

Mărimea	Ecuații	Agenți, fără spațiu	Agenți, în spațiu
Primul vârf (infecțioși)	3 608	5 224	3 229
Ziua primului vârf	46	45	46
Minimul dintre valuri (zilele 60–140)	≈ 1 364 (echilibru)	1	23
Comportament pe termen lung	echilibru endemic	valuri, aproape de stingere	valuri locale, focare

- Ecuațiile presupun durate **exponențiale**; agenții au durate uniforme, mai regulate, deci vârfuri mai ascuțite
- Spațiul **încetinește** saturarea: vecinii unui bolnav sunt deja infectați sau imuni, iar focarele locale supraviețuiesc între valuri
- Stingerea completă (0 infecțioși) este posibilă doar într-un model cu indivizi – ecuațiile nu ajung niciodată la zero

Ce câștigăm și ce plătim

- Câștig: **eterogenitate** – vârstă, comportament, număr de contacte diferite pentru fiecare persoană
- Câștig: **spațiu și rețele** – focare locale, fronturi, rolul transportului și al școlilor
- Câștig: **evenimente rare** – stingere, superrăspânditori, focare care pornesc dintr-un singur caz
- Plată: **calcul** – mii de rulări pentru o analiză completă
- Plată: **calibrare** – mulți parametri, adesea fără date individuale; risc de modele „cu prea multe butoane”

H **Holban**

Nu cunoaștem ecuațiile globale care descriu o epidemie într-un spațiu dat și într-un context dat – de aceea se folosesc modele cu agenți cu stări.

Verificare în practică: două neconcordanțe în textul original

- Textul spune că un contact transmite boala cu probabilitatea **0,1**, iar mai jos că șansa este de **0,1%** – de 100 de ori mai mică
- Cu 0,1%: $R_0 = 5 \times 0{,}001 \times 11 \approx$ **0,055** – epidemia nu pornește deloc
- Spațiul de 10 × 10 km este descris ca o matrice 1 000 × 1 000 cu celule de 100 × 100 m; corect: **100 × 100** de celule (sau celule de 10 m)
- Noi folosim p = 0,1 și coordonate continue – rezultatele au sens epidemiologic
- Astfel de neconcordanțe apar des: verificarea înseamnă să recalculăm, nu doar să citim

V **Obicei de inginer**

Înainte de a rula un model, calculați „pe hârtie” o mărime cheie (aici R_0). Dacă ea nu are sens, modelul nu are sens.

Verificare

? În modelul Holban, un infectat are **5** contacte pe zi, transmisie **0,1** pe contact și boala durează în medie **11** zile. Cât este, aproximativ, R_0?

A 0,5

B 1,1

C 5,5

D 55

Agenți care învață

Problema barului El Farol

1. Arthur (1994): raționament inductiv
2. Predictori, încredere, adaptare
3. Diversitatea strategiilor contează

Barul El Farol (Arthur, 1994)

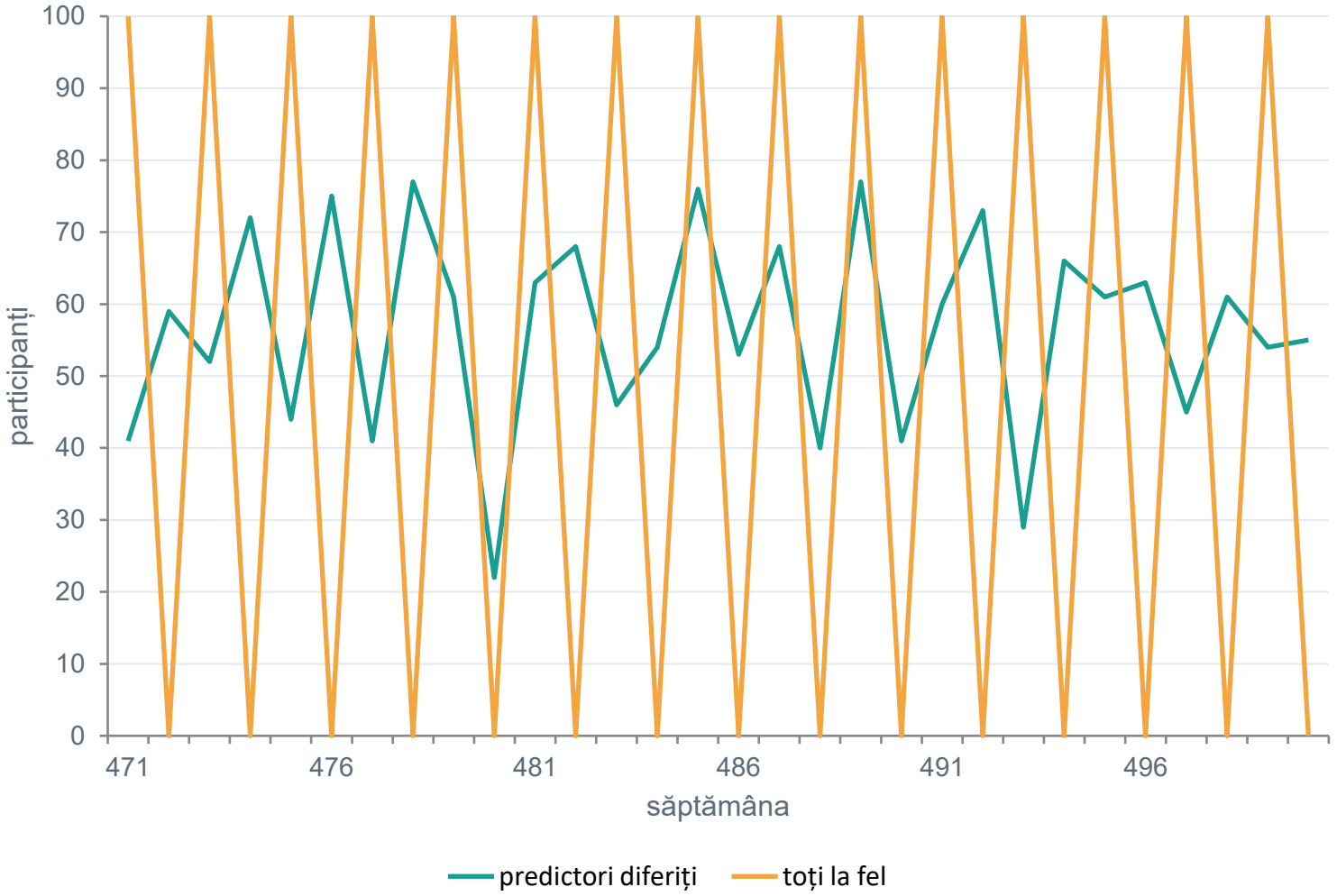
100 de oameni, un bar plăcut doar dacă vin cel mult 60

În fiecare săptămână, fiecare agent decide independent dacă merge la bar. Dacă vin cel mult 60, toți cei prezenți se distrează; dacă vin mai mulți, ar fi fost mai bine acasă. Nu există comunicare: singura informație este **istoricul** prezenței.

W. B. Arthur, Inductive reasoning and bounded rationality, American Economic Review, 1994

- Fiecare agent are câțiva **predictori**: „ca săptămâna trecută”, „media ultimelor 4 săptămâni”, „opusul săptămânii trecute”…
- Folosește predictorul care a greșit **cel mai puțin** până acum (încrederea scade cu eroarea)
- Merge dacă predicția este ≤ 60
- Nu există o strategie „corectă” comună: dacă toți ar prezice la fel, toți ar greși

Prezența la bar: predictori diferiți vs predictori identici



- Predictori diferiți: medie **57,6**, abatere 14,7 – aproape de capacitatea de 60
- Toți la fel: 100 sau 0 participanți, alternativ (medie 50)
- Nimeni nu știe capacitatea „optimă” – sistemul o găsește singur
- Ultimele 30 din 500 de săptămâni simulate

Ce ne învață El Farol

- **Raționalitate limitată**: agenții nu rezolvă un echilibru matematic, ci folosesc reguli simple și le ajustează
- **Ecologie de predictori**: predictorii buni câștigă încredere până când prea mulți îi folosesc – și încep să greșească
- Prezența medie se **auto-organizează** în jurul capacității, fără coordonare
- Variantă simplificată: **jocul minorității** (Challet și Zhang, 1997) – câștigă cei din grupul mai mic
- Aplicații: alegerea rutelor în trafic, a momentului de acces la un server, strategii pe piețe

A **De ce agenți?**

Un model cu ecuații ar presupune un „agent reprezentativ”. Aici tocmai **diferențele** dintre agenți fac sistemul să funcționeze.

Verificare

? De ce, când **toți** agenții folosesc același predictor, barul El Farol este alternativ plin (100) și gol (0)?

A pentru că predictorul este greșit

B pentru că toți primesc aceeași predicție și iau aceeași decizie, care își anulează propria premisă

C pentru că agenții nu au memorie

D din cauza zgomotului aleator

Când merită un model bazat pe agenți?

- Interacțiunile sunt **discrete**, locale sau pe o rețea, iar structura lor contează
- Indivizii sunt **eterogeni** și diferențele schimbă rezultatul (vârstă, strategie, poziție)
- Agenții **învață** sau se **adaptează**: comportamentul se schimbă în timp
- Ne interesează fenomene **emergente** sau evenimente rare, nu doar medii
- Datele sau intuiția există la nivel **individual** (reguli de comportament), nu la nivel agregat

N **Când nu**

Dacă populația este mare, omogenă și bine amestecată, iar ne interesează doar mediile, ecuațiile sunt mai rapide, mai transparente și mai ușor de calibrat.

Rezumatul orei 1

- ✓ Agentul: entitate **autonomă** care percepe, decide și acționează; reactiv, cu stare, orientat pe scop/utilitate, care învață
- ✓ Sistem multi-agent: mediu (grilă, spațiu continuu, rețea), interacțiuni directe, prin mediu sau globale
- ✓ Actualizarea **sincronă** poate produce artefacte; asincron cu ordine amestecată
- ✓ Epidemia Holban: spațiul coboară vârful (3 229 vs 5 224) și produce valuri; două erate în text
- ✓ El Farol: agenți care învață se auto-organizează la capacitate doar dacă sunt **diferiți**

Ora 2

Modele clasice

1. Boids: stoluri din trei reguli
2. Schelling: segregare fără intenție (demo-ul săptămânii)
3. Furnici, cadavre și bacterii (Cernazean L8)
4. SIR cu agenți, lume mică și rețele fără scară

Ce au în comun modelele clasice

- Agenți **simpli**, cu 2–4 reguli care se pot spune într-o propoziție
- Percepție **locală**: vecinii apropiați, celula curentă, memoria ultimilor pași
- Un fenomen **colectiv** care nu apare în regulile individuale: stol, cartier, grămadă
- Un parametru care schimbă calitativ comportamentul: prag de toleranță, pondere, probabilitate
- Sunt „modele de jucărie” în sensul bun: explică **mecanisme**, nu prezic cifre exacte

M **Cum le folosim**

Un model clasic este un argument: „acest mecanism simplu este suficient pentru a produce fenomenul”. Nu demonstrează că mecanismul este singura cauză reală.

Reynolds, 1987

Boids: stoluri din trei reguli

1. Separare, aliniere, coeziune
2. Implementare vectorizată
3. Parametrul de ordine
4. De la Batman la grauri și drone

Modelul Boids

Fiecare „pasăre” urmează trei reguli locale

Fiecare boid vede doar vecinii aflați într-o **rază** dată (și, opțional, într-un **unghi** de vedere în direcția de zbor). Viteza se ajustează în fiecare pas după trei reguli, apoi boidul înaintează. Nu există lider și nici plan global.

C. Reynolds, Flocks, herds and schools: a distributed behavioral model, SIGGRAPH 1987; Holban, cap. 7

- **Separare**: evită coliziunile cu vecinii prea apropiați
- **Aliniere**: își potrivește viteza cu viteza medie a vecinilor
- **Coeziune**: se deplasează spre centrul vecinilor
- Holban: agent **reactiv**; mediul este spațiul 2D sau 3D, eventual cu obstacole

Cele trei reguli, pe rând

1 Separare

Raza mică (2,5): fiecare vecin prea apropiat împinge boidul în direcția opusă, cu o forță ~ 1/distanță. Fără ea, stolul se prăbușește într-un punct.

2 Aliniere

Raza mare (10): boidul își rotește viteza spre viteza medie a vecinilor. Ea creează mișcarea **ordonată**, în aceeași direcție.

3 Coeziune

Raza mare (10): boidul este atras spre centrul vecinilor. Ea ține grupul **împreună** și unește grupuri mici în grupuri mari.

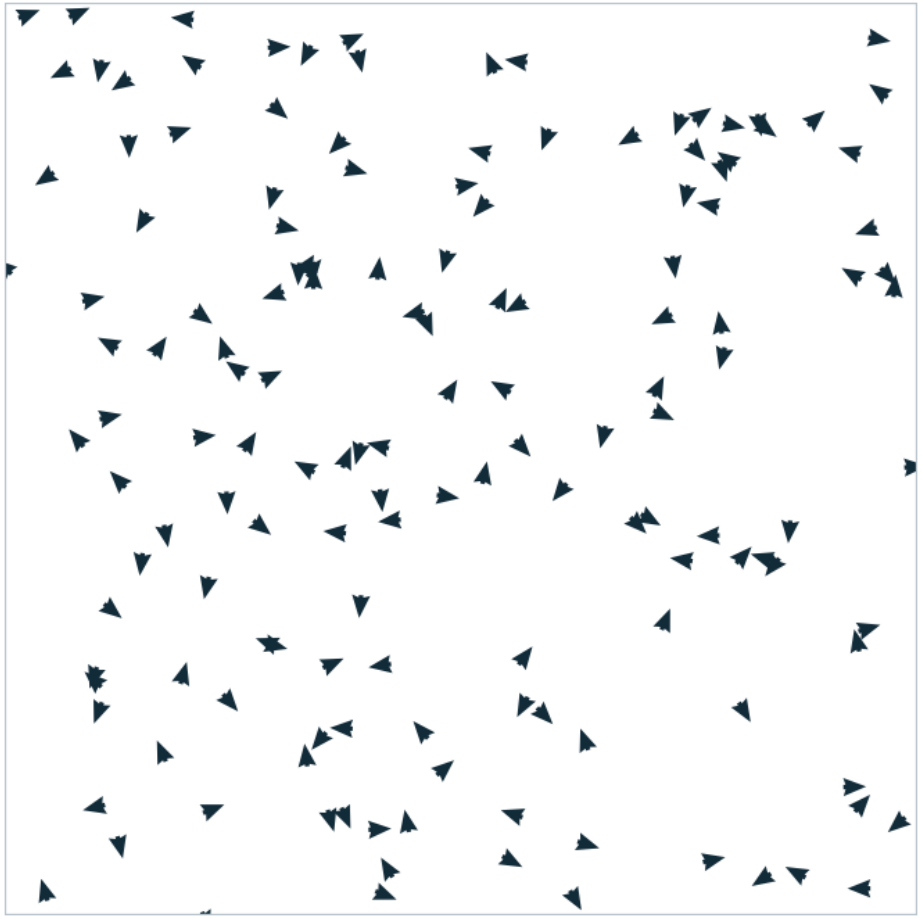
În implementarea noastră: 150 de boizi într-un spațiu periodic 100 × 100, viteză constantă 1, ponderi 0,15 (separare), 0,05 (aliniere), 0,005 (coeziune) și un mic zgomot. Raportul dintre ponderi decide aspectul stolului.

Un pas Boids, vectorizat cu NumPy

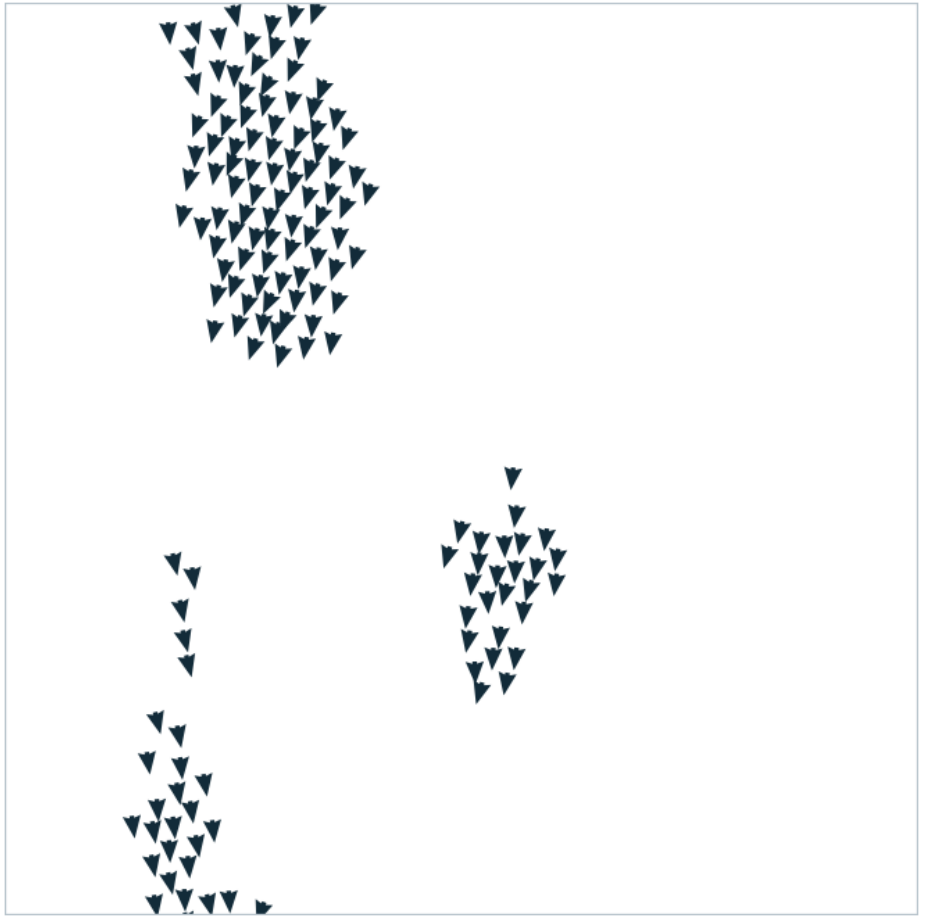
```
d = X[None, :, :] - X[:, None, :]
d -= box * np.round(d / box)          # distanța pe tor
dist = np.hypot(d[..., 0], d[..., 1])
np.fill_diagonal(dist, np.inf)
nb = dist < 10; k = np.maximum(nb.sum(1, keepdims=True), 1)
ali = (nb @ V) / k - V                 # aliniere
coh = (nb[..., None] * d).sum(1) / k   # coeziune
close = dist < 2.5
sep = -(close[..., None] * d / dist[..., None]**2).sum(1)
V = V + 0.05 * ali + 0.005 * coh + 0.15 * sep
V /= np.linalg.norm(V, axis=1, keepdims=True)
X = (X + V) % box                      # spațiu periodic
```

- Toate perechile odată: matrice N × N (S8)
- nb: cine vede pe cine
- Viteza se normalizează: boizii nu accelerează
- Bucle Python vs NumPy, N = 300: 16× mai lent

150 de boizi: starea inițială și după 600 de pași

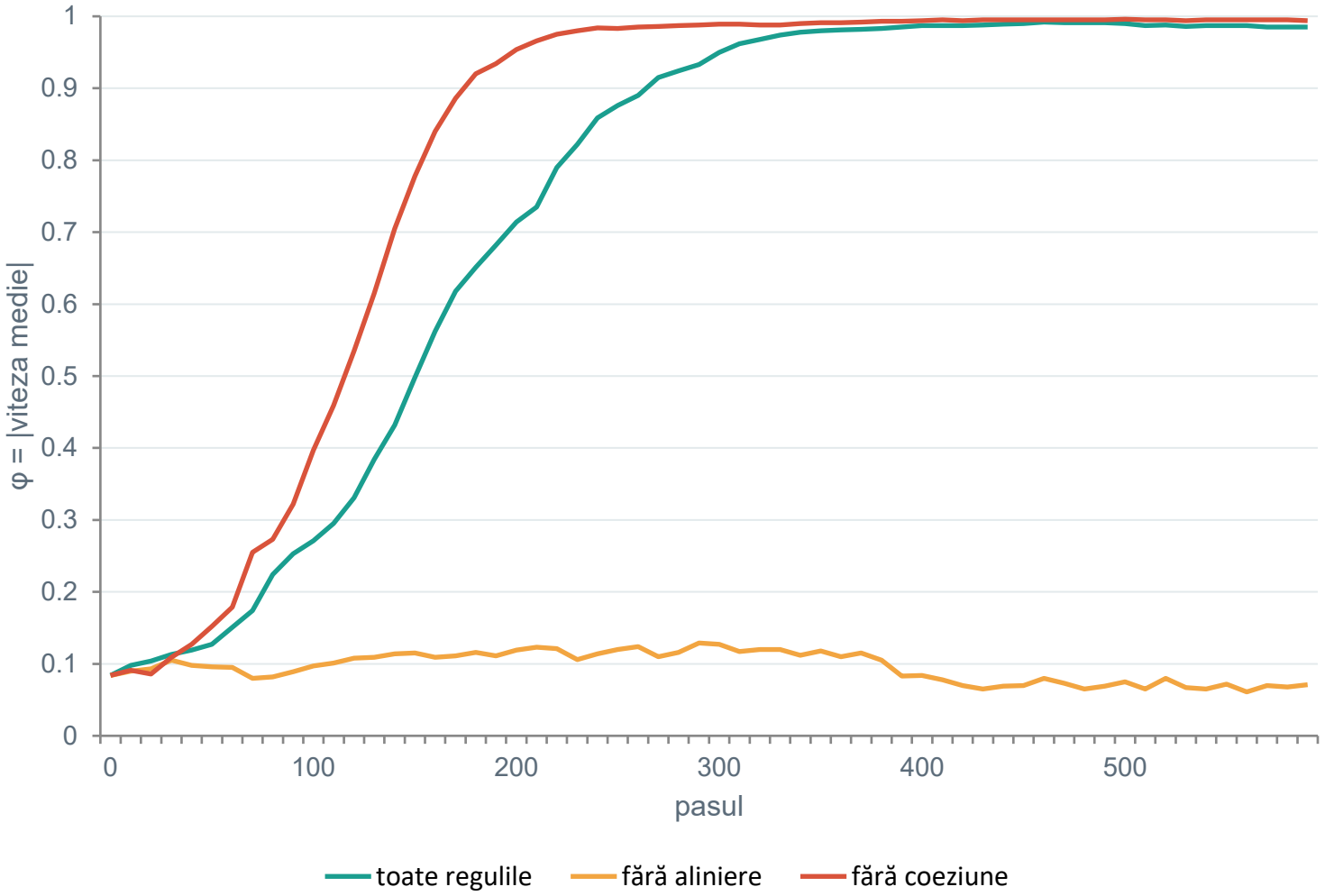


pasul 0: direcții aleatoare, ordine $\phi = 0{,}08$



pasul 600: 2 stoluri compacte, aproape aliniate ($\phi = 0{,}99$)

Parametrul de ordine: cât de aliniat este stolul



- ϕ = lungimea vitezei medii: 0 = haos, 1 = toți în aceeași direcție
- Toate regulile: $\phi \approx$ **0,99** după ~200 de pași
- Fără aliniere: $\phi \approx 0{,}07$ – grupuri, dar fără direcție comună
- Fără coeziune: tot aliniați, dar împrăștiați în 5 grupuri

Boids în lumea reală

- **Film**: liliecii și pinguinii din Batman Returns (1992); Reynolds a primit în 1998 un premiu tehnic al Academiei Americane de Film
- **Biologie**: graurii urmăresc ~6–7 vecini (Ballerini et al., 2008); bancurile de pești, roiurile de lăcuste
- **Fizică**: modelul Vicsek (1995) – doar aliniere și zgomot; tranziție de fază de la haos la ordine
- **Robotică**: roiuri de drone și roboți care zboară în formație fără control central
- **Jocuri**: mulțimi, armate, bancuri de pești – aceleași trei reguli, cu obstacole și ținte

L **Laborator**

Implementați Boids în laboratorul 5–6; încercați să adăugați un prădător sau obstacole și să măsurați ϕ.

Verificare

? Eliminăm regula de **aliniere** din Boids și păstrăm separarea și coeziunea. Ce se întâmplă cu parametrul de ordine ϕ?

A crește spre 1

B rămâne mic: grupuri fără direcție comună

C devine exact 0

D oscilează între 0 și 1

Schelling, 1971

Segregare fără intenție

1. Regula: mută-te dacă ai prea puțini vecini ca tine
2. Demo: pragul de toleranță 30%
3. Pragul și efectele lui
4. Ce dovedește și ce nu

Modelul de segregare Schelling

Pragul t: „vreau ca măcar o fracțiune t din vecini să fie ca mine”

O grilă de case, ocupate de agenți din două grupuri (sau goale). Un agent este **mulțumit** dacă fracțiunea vecinilor din grupul lui (din cei 8 vecini ocupați) este cel puțin t. Agenții nemulțumiți se mută într-o casă goală aleasă la întâmplare.

T. C. Schelling, Dynamic models of segregation, Journal of Mathematical Sociology, 1971

- Grilă 50 × 50, 90% ocupată, două grupuri egale
- Actualizare asincronă, în ordine aleatoare, până nu se mai mută nimeni
- Cu t = 0,3, un agent acceptă să fie în minoritate: 3 vecini similari din 8 îi ajung
- Măsura segregării: fracțiunea medie de vecini din același grup

Demo: Schelling în Mesa 3 (Python)

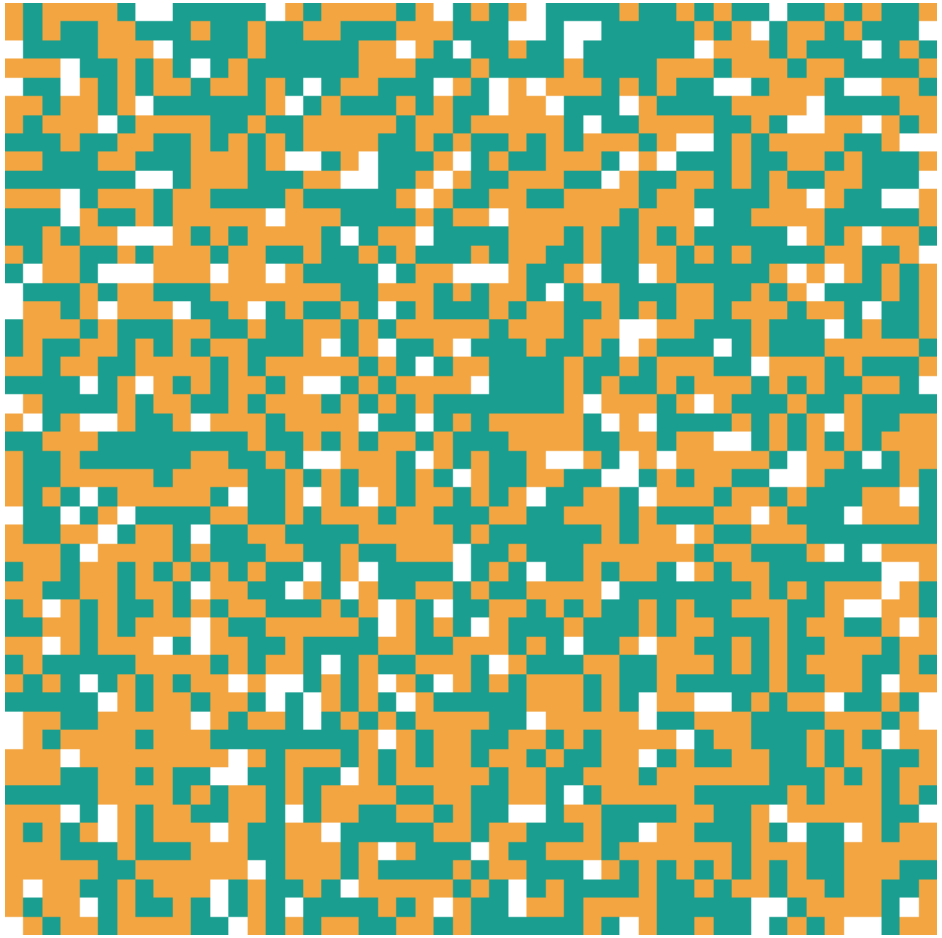
```
import mesa

class Locuitor(mesa.Agent):
    def __init__(self, model, grup):
        super().__init__(model)
        self.grup = grup
    def step(self):
        vec = self.model.grid.get_neighbors(self.pos, moore=True)
        la_fel = sum(v.grup == self.grup for v in vec)
        if vec and la_fel / len(vec) < self.model.prag:
            self.model.grid.move_to_empty(self)

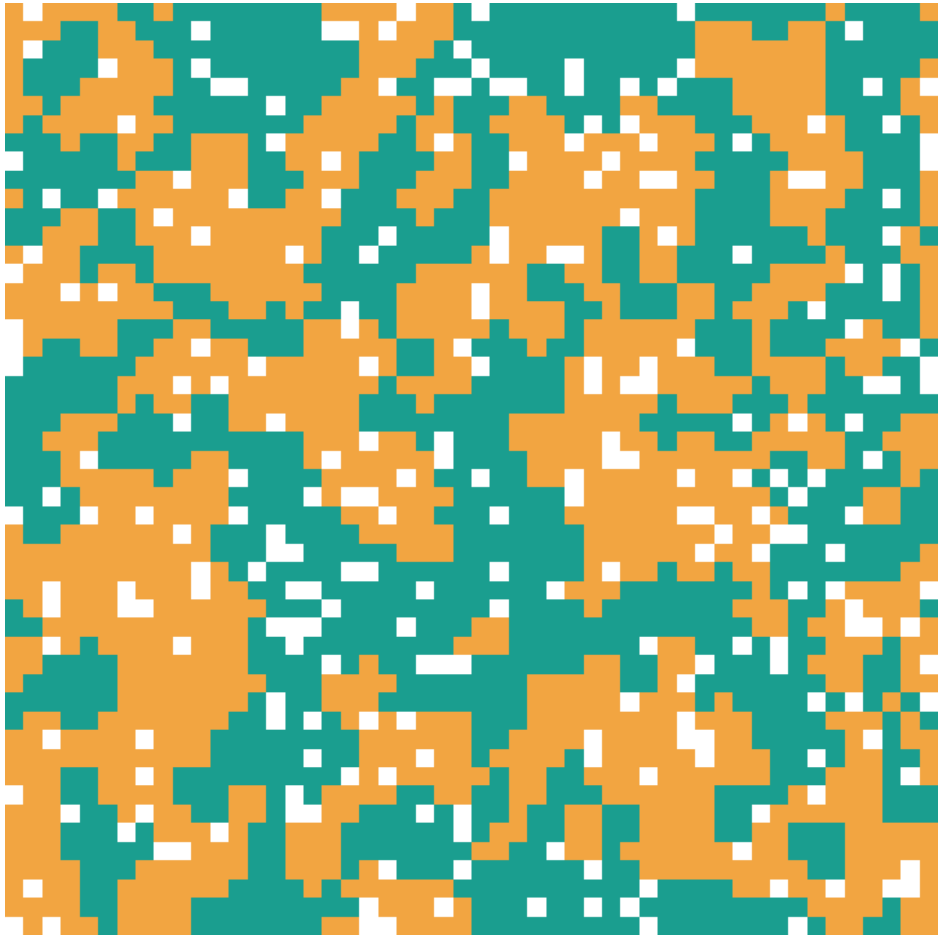
class Schelling(mesa.Model):
    def __init__(self, n=50, densitate=0.9, prag=0.3, seed=None):
        super().__init__(seed=seed)
        self.prag = prag
        self.grid = mesa.space.SingleGrid(n, n, torus=True)
        for _, pos in self.grid.coord_iter():
            if self.random.random() < densitate:
                a = Locuitor(self, self.random.random() < 0.5)
                self.grid.place_agent(a, pos)
    def step(self):
        self.agents.shuffle_do("step")
```

- Clasa Agent = comportamentul unui locuitor
- Clasa Model = grila, parametrii, pasul
- shuffle_do: actualizare asincronă în ordine aleatoare
- Testat cu Mesa 3.3: segregare 0,74 cu prag 0,3

Demo: pragul de 30% produce cartiere separate

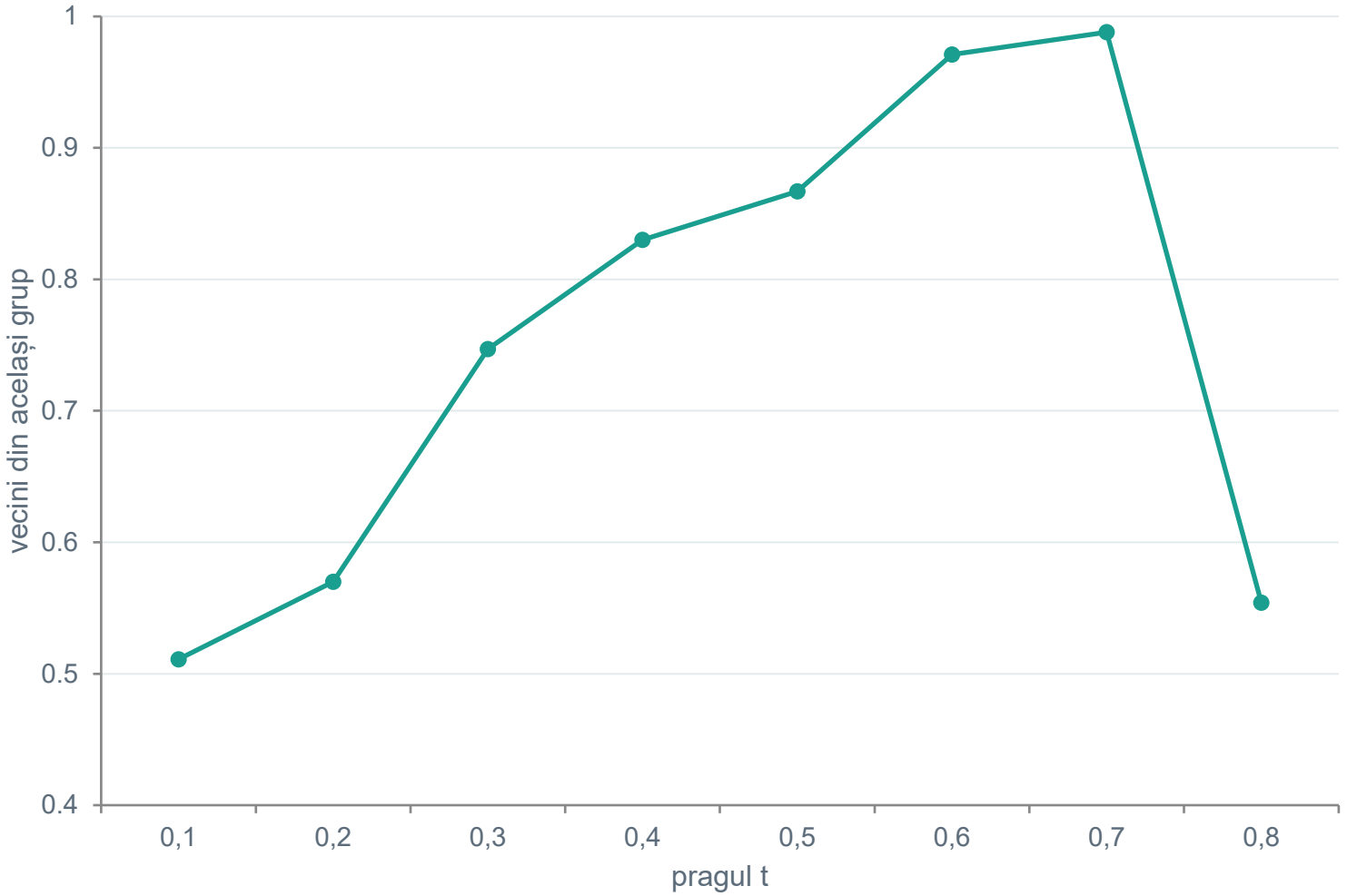


start: amestec aleator, 50% vecini din același grup



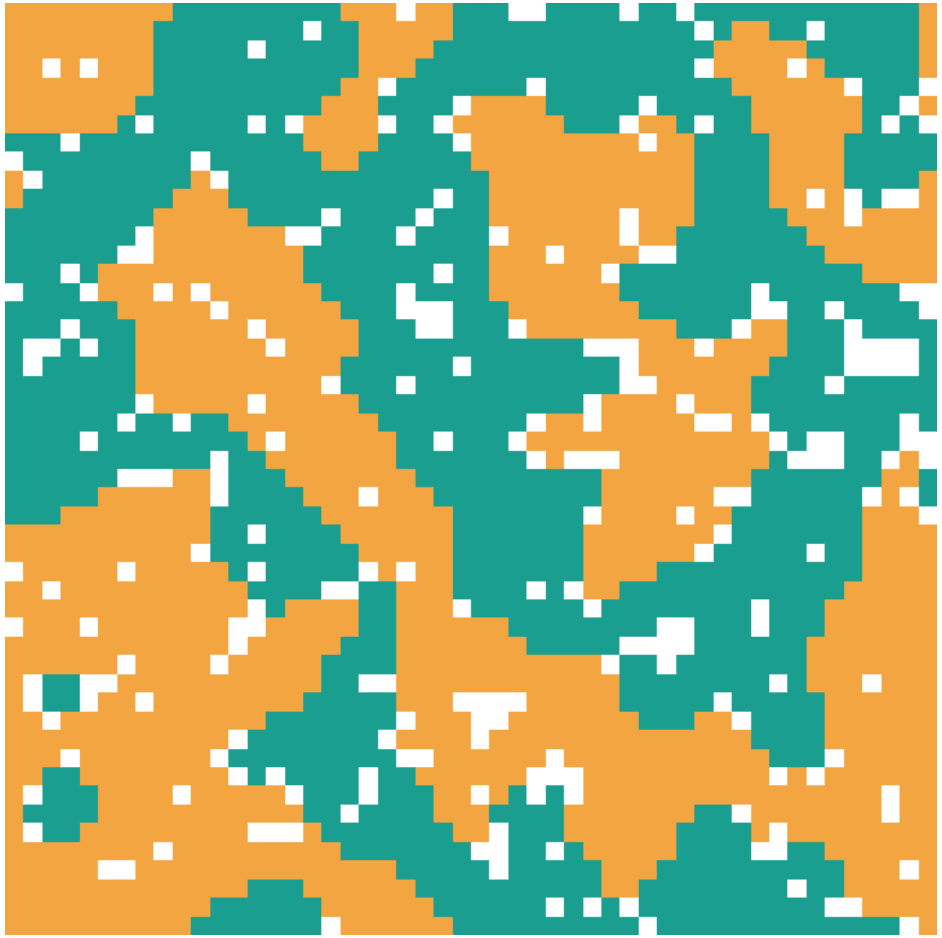
după 11 runde: **74%** vecini din același grup

Segregarea finală în funcție de pragul de toleranță



- t = 0,3: 75%; t = 0,5: 87%
- t = 0,7: 99% – cartiere aproape pure
- t = 0,8: nu există echilibru; 85% rămân nemulțumiți, amestec 55%
- Medie pe 3 rulări pentru fiecare prag

Praguri mai mari: segregare aproape completă



t = 0,5: 88% vecini din același grup, 22 runde



t = 0,75: 99%, după 151 de runde

Ce arată Schelling și ce nu

- **Micromotive și macrocomportament** (Schelling, 1978): preferințe individuale slabe pot produce rezultate colective extreme
- Nu putem deduce **intențiile** indivizilor din tiparul global: segregarea puternică nu implică preferințe puternice
- Modelul **nu** dovedește că segregarea reală are această cauză – prețuri, discriminare, politici joacă și ele un rol
- Extensii: mai multe grupuri, prețuri ale caselor, rețele sociale, date reale de recensământ
- Schelling a primit Premiul Nobel pentru economie în 2005 (pentru teoria jocurilor și conflict)

S **Lecția metodologică**

Un model simplu poate **infirma** o explicație intuitivă („segregarea arată că oamenii vor segregare”), chiar dacă nu oferă explicația completă.

Verificare

? În demo-ul Schelling, fiecare agent acceptă să fie în minoritate (prag **30%**). Ce fracțiune medie de vecini din același grup obținem la final?

A în jur de 30%

B în jur de 50%, ca la început

C în jur de 75%

D 100%: segregare totală

Cernazean L8

Furnici și bacterii

1. Grămezile de cadavre ale furnicilor Messor sanctus
2. Modelul Deneubourg (1991)
3. Chemotaxia: alergare și rostogolire

Observația: furnicile Messor sanctus fac „cimitire”

- Experiment (Jost et al., 2007): cadavre de furnici împrăștiate uniform într-o arenă cu muncitoare vii
- După câteva ore apar grămezi mici; după 24 de ore rămân **câteva grămezi mari**
- Nicio furnică nu are o hartă și nu există un „șef” care să decidă unde se face grămada
- Întrebările lui Cernazean: cum funcționează? Este „inteligență de roi”? Care este **cel mai simplu** model care explică procesul?
- Aceeași logică la sortarea larvelor după mărime și la roboții care strâng obiecte

M **Strategia de modelare**

Pornim de la cel mai simplu agent imaginabil și adăugăm „inteligență” doar dacă modelul simplu nu reproduce tiparul observat.

Modelul Deneubourg (1991): când ridică și când lasă o furnică

$$Pp = (k_1 / (k_1 + f))^2 \qquad Pd = (f / (f + k_2))^2$$

f	fracțiunea de celule cu cadavre întâlnite în ultimii n pași (memoria furnicii); mare = zonă aglomerată
Pp	probabilitatea de a **ridica** un cadavru: mare dacă f e mic (cadavru izolat)
Pd	probabilitatea de a **lăsa** cadavrul purtat: mare dacă f e mare (lângă alte cadavre)

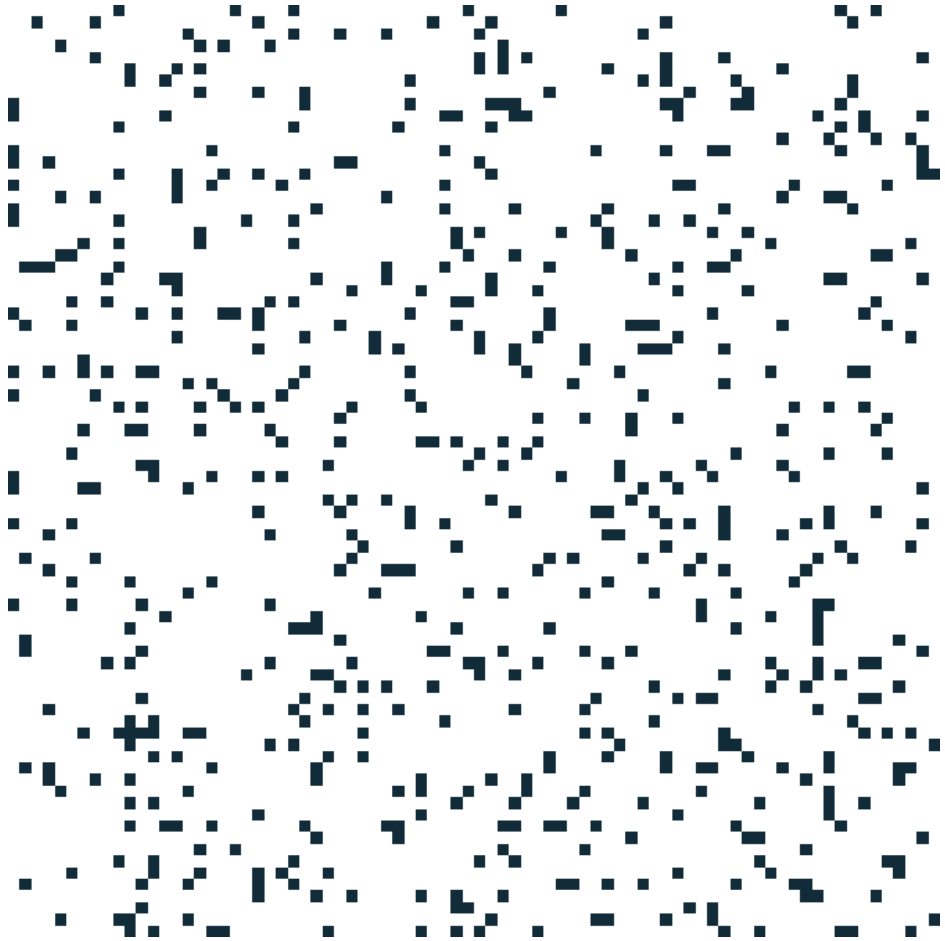
Furnicile fac pași aleatori pe o grilă cu 4 direcții, actualizate secvențial. Noi folosim $k_1 = 0{,}1$, $k_2 = 0{,}3$ și o memorie de n = 20 de pași. Atenție: în slide-urile originale f apare ca suma memoriei, nu ca fracțiune – cu suma, valorile k_1 și k_2 nu mai sunt comparabile.

O furnică, un pas

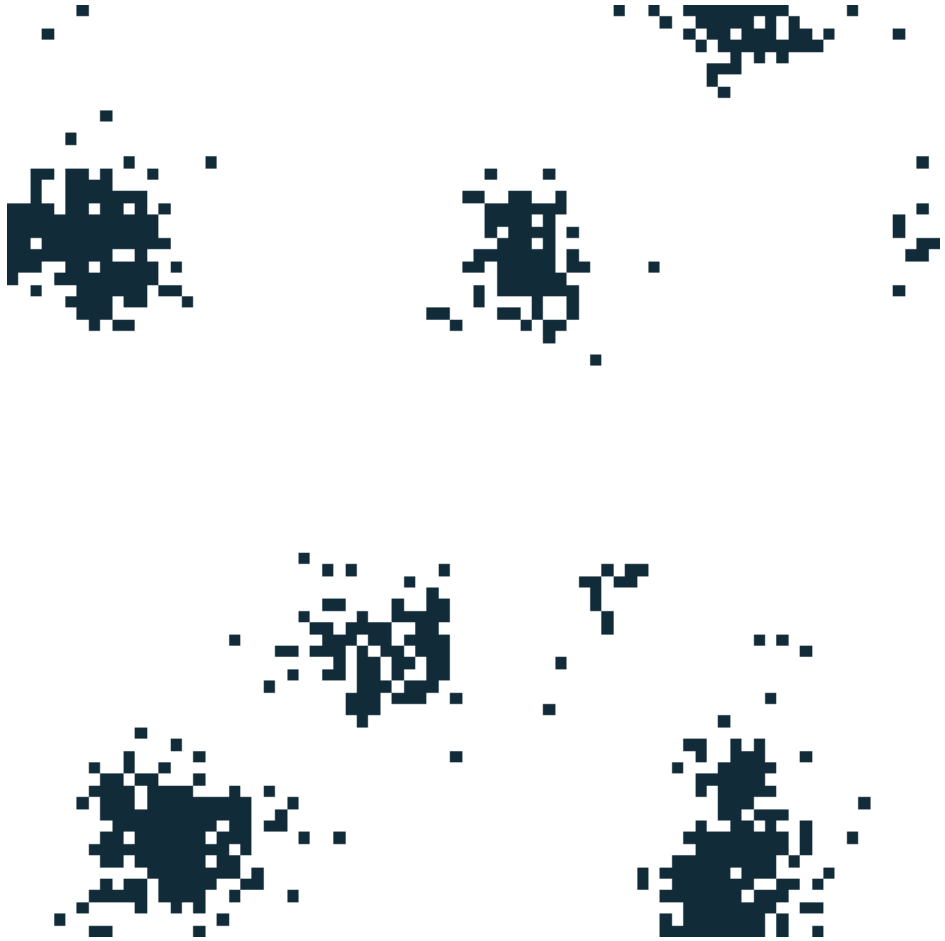
```
i, j = (i + di) % n, (j + dj) % n       # pas aleator
aici = grila[i, j]
memorie[k] = aici; k = (k + 1) % 20      # ultimii 20 de pași
f = sum(memorie) / 20
if not incarcata and aici:
    if rng.random() < (k1 / (k1 + f))**2:
        grila[i, j] = 0; incarcata = True
elif incarcata and not aici:
    if rng.random() < (f / (f + k2))**2:
        grila[i, j] = 1; incarcata = False
```

- Agent cu **stare**: încărcată sau nu, plus memoria
- Interacțiune doar **prin mediu**: grila de cadavre (stigmergie)
- Grilă 80 × 80, 640 de cadavre, 40 de furnici
- 400 000 de pași × 40 de furnici: ~23 s în Python

Cadavrele: la început și după 400 000 de pași

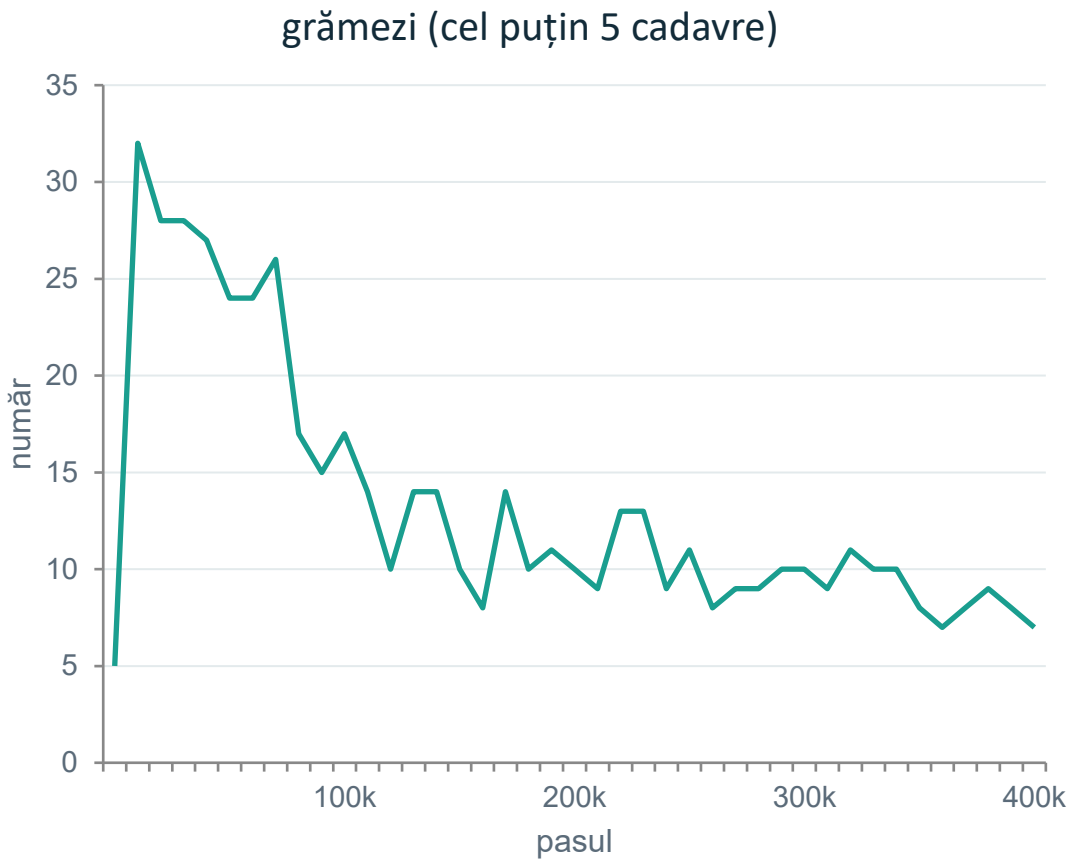


start: 640 de cadavre împrăștiate aleator

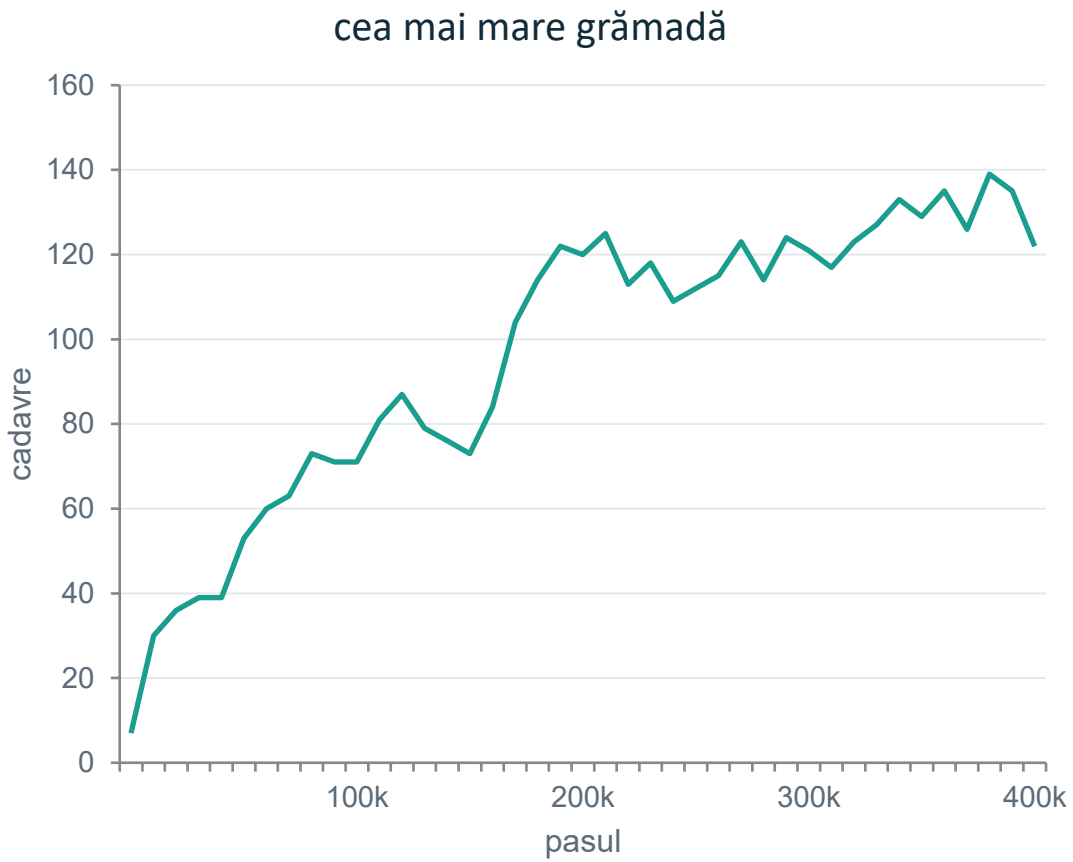


după 400 000 de pași: 7 grămezi, cea mai mare cu 122 de cadavre

Cum se formează grămezile



întâi apar multe grămezi mici, apoi numărul lor scade



grămezile mari cresc pe seama celor mici

Furnici „deștepte” și furnici „proaste” (Cernazean)

- Modelul Deneubourg cere furnici destul de „inteligente”: memorie și estimarea densității
- Modelul Unige: furnica **ridică mereu** un cadavru găsit și îl **lasă mereu** lângă altul – fără memorie
- Și furnicile „proaste” formează grămezi: mecanismul de bază sunt **fluctuațiile statistice**
- Inteligența doar **accelerează** procesul: ~10 ori mai rapid la Deneubourg
- Nu e un efect colectiv: timpul scade proporțional cu numărul de furnici – o singură furnică ar reuși, dar mai încet

S **Stigmergie**

Grassé (1959): coordonare prin urmele lăsate în mediu. Aceeași idee apare la termite, la potecile din parcuri și în algoritmii de optimizare cu colonii de furnici.

Chemotaxia: cum găsește o bacterie hrana

Alergare și rostogolire, cu o memorie de un singur număr

Bacteria E. coli înoată în linie dreaptă (**alergare**), apoi se rostogolește și pleacă într-o direcție aleatoare (**rostogolire**). Nu poate „vedea” gradientul, dar își amintește concentrația de nutrient de acum o clipă.

Cernazean L8; Alon et al., Robustness in bacterial chemotaxis, Nature 397, 1999

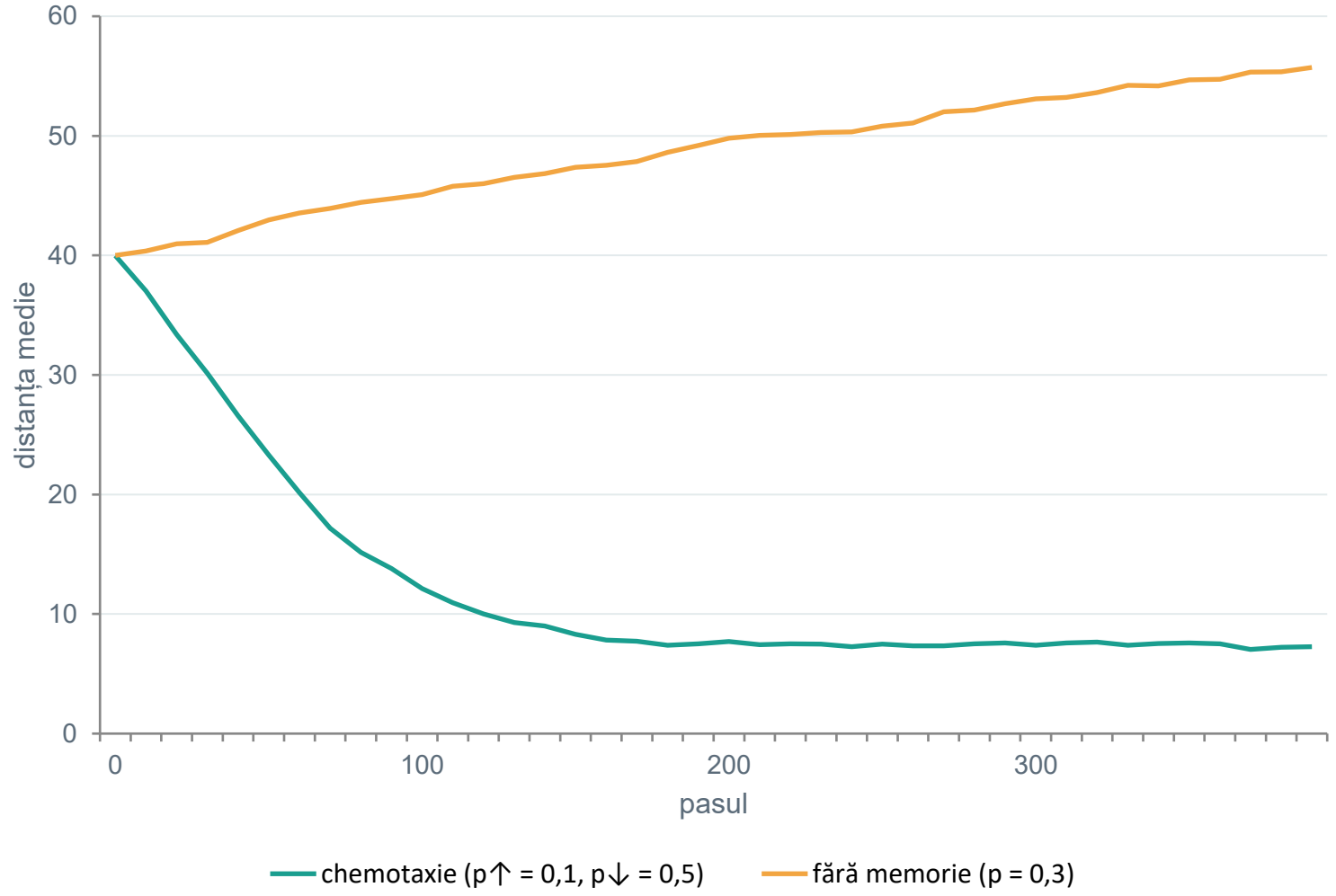
- Dacă concentrația **crește**, se rostogolește rar (probabilitate p↑ mică)
- Dacă concentrația **scade**, se rostogolește des (p↓ mare)
- Model Cernazean: grilă euleriană cu nutrient care difuzează; starea bacteriei = (ultima direcție, ultima concentrație)
- Mecanismul molecular: o rețea de proteine care compară concentrația actuală cu cea recentă

Chemotaxia în 10 linii

```
conc = lambda X: np.exp(-np.hypot(X[:, 0], X[:, 1]) / 30)
m = conc(X)                              # memoria: o valoare
for t in range(400):
    c = conc(X)
    p = np.where(c > m, 0.1, 0.5)  # crește: rar; scade: des
    m = c
    rost = rng.random(N) < p
    th[rost] = rng.random(rost.sum()) * 2 * np.pi
    X = X + np.c_[np.cos(th), np.sin(th)]   # alergare
```

- Sursa de hrană în origine; concentrația scade cu distanța
- Starea fiecărei bacterii: direcția th și memoria m
- Varianta „fără memorie”: p = 0,3 indiferent de c
- Cernazean: aceeași regulă, pe o grilă cu nutrient care difuzează

Distanța medie până la sursa de hrană



- 300 de bacterii pornesc la distanța 40 de sursă
- Cu chemotaxie: distanța scade la **7,3**
- Fără memorie: 55,3 – se împrăștie
- La final, în raza 10: 74% vs 2%

Verificare

? Ce informație minimă îi trebuie unei bacterii pentru chemotaxie în modelul prezentat?

A harta completă a concentrației

B direcția gradientului, măsurată la capetele celulei

C concentrația de la pasul anterior, pentru comparație

D pozițiile celorlalte bacterii

Epidemii și rețele

SIR cu agenți și rețele complexe

1. Ce câștigăm față de ecuații
2. Lumea mică: Watts–Strogatz
3. Rețele fără scară: Barabási–Albert
4. Epidemii și vaccinare pe rețele

SIR cu ecuații vs SIR cu agenți

Ecuații (S7)

- o singură rată β pentru toată populația
- $R_0 = \beta/\gamma$ decide totul
- mărimea finală depinde doar de R_0
- deterministe: aceleași rezultate mereu
- nu ajung niciodată exact la zero infectați

vs

Agenți

- fiecare agent are contactele lui (rețea, spațiu)
- contează **cine** cu cine: noduri centrale, clustere
- superrăspânditori, focare locale
- stocastice: stingere timpurie posibilă
- măsuri țintite: carantină, vaccinarea huburilor

Același R_0 mediu poate produce epidemii foarte diferite dacă distribuția contactelor este diferită. Rețeaua de contacte devine o parte a modelului, la fel de importantă ca parametrii bolii.

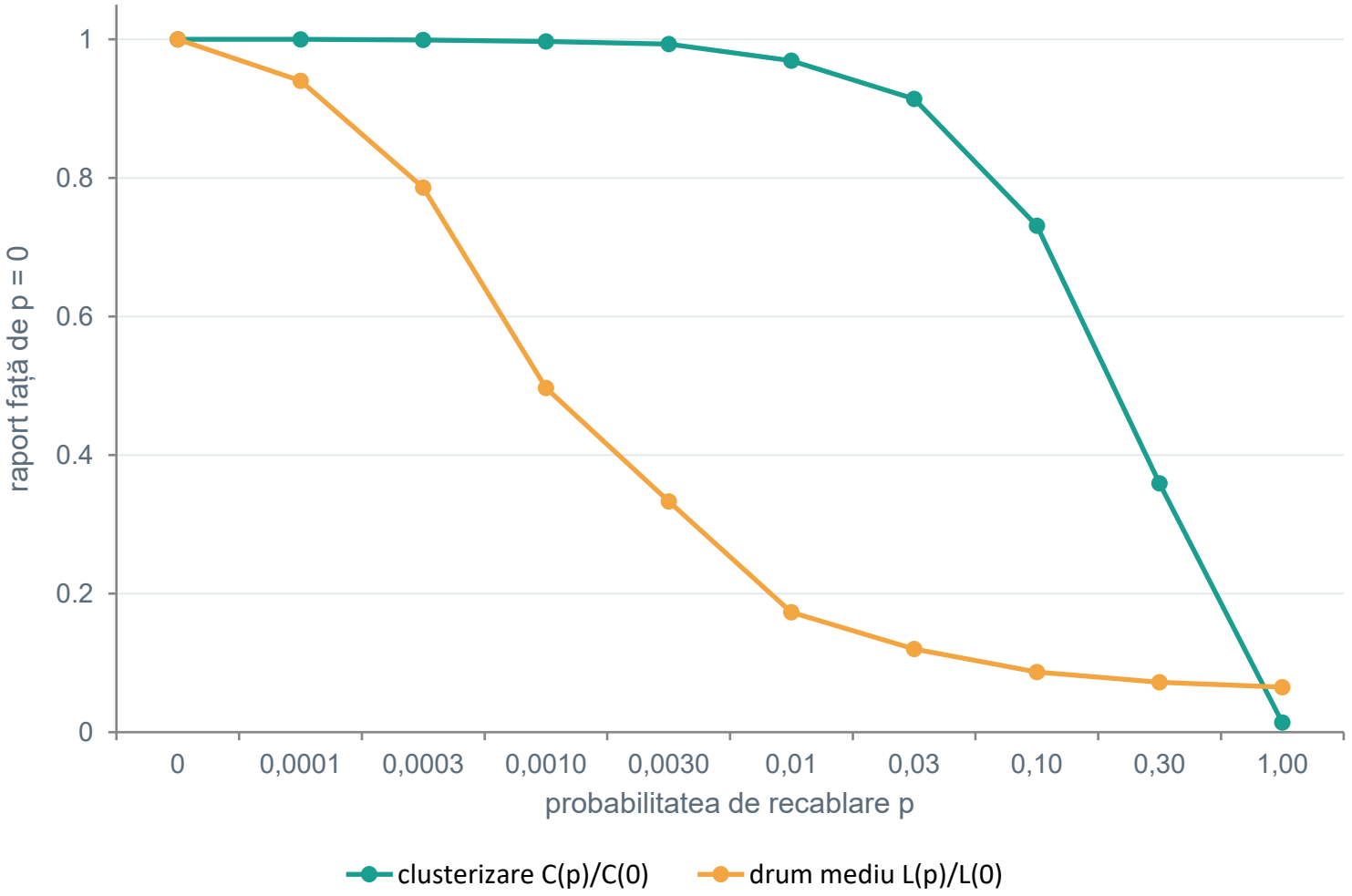
Rețele complexe: lumea mică și rețelele fără scară

- **Șase grade de separare**: experimentul cu scrisori al lui Milgram (1967); pe Facebook, 3,57 grade în medie (2016)
- **Watts și Strogatz (1998)**: puține legături aleatoare într-o rețea regulată scurtează dramatic drumurile, păstrând grupurile locale
- **Barabási și Albert (1999)**: atașament preferențial – nodurile noi se leagă de preferință la cele deja populare
- Rezultat: distribuție a gradelor cu „coadă lungă” – câteva noduri foarte conectate (**huburi**)
- Exemple: web, citări, rețele sociale, aeroporturi, interacțiuni între proteine

R **Pentru informaticieni**

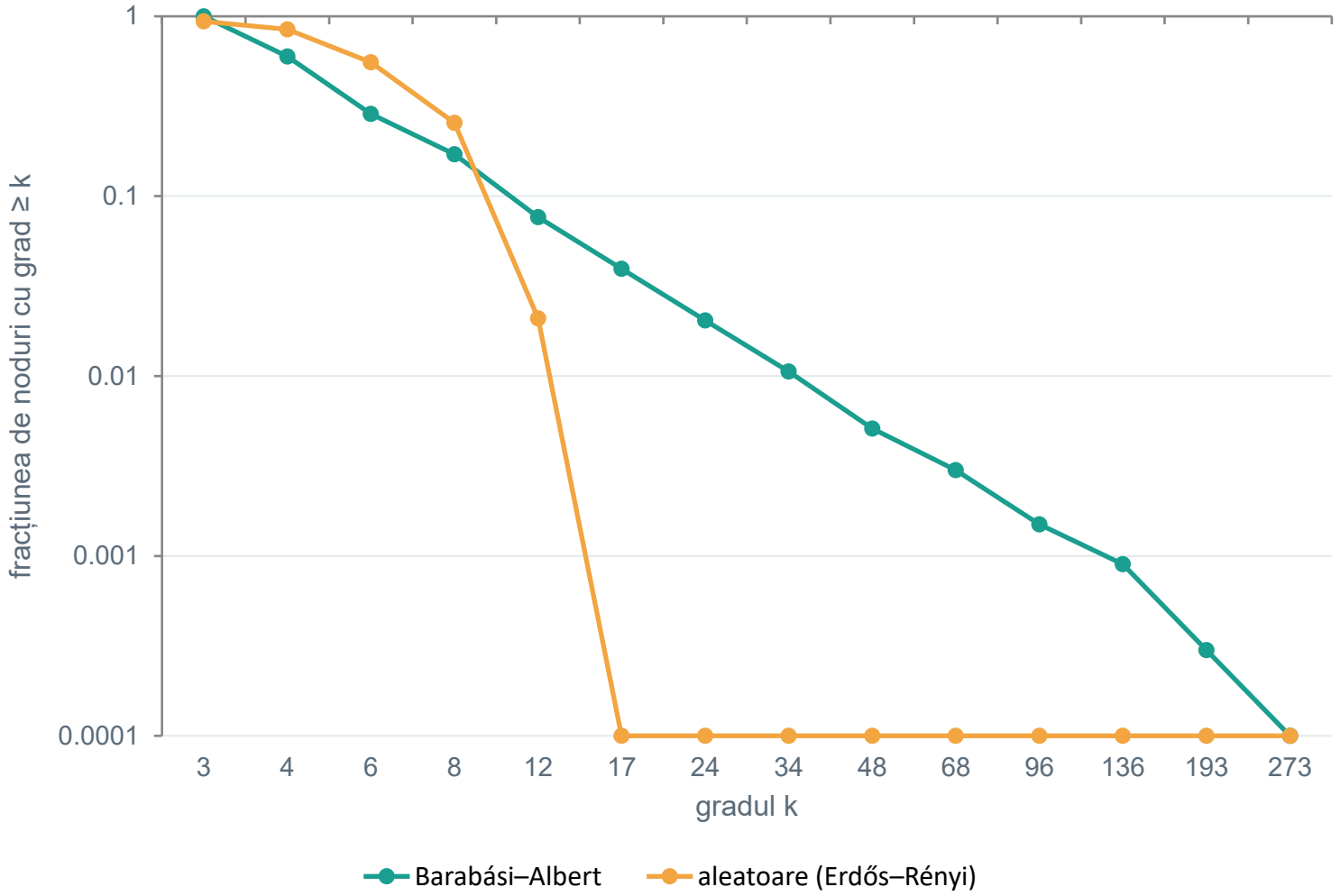
Aceleași modele de rețea se folosesc pentru a testa algoritmi distribuiți, rutarea, robustețea internetului și răspândirea programelor malițioase.

Lumea mică: câteva legături aleatoare scurtează drumurile



- 1 000 de noduri, câte 10 vecini; L(0) = 50,5 pași
- p = 0,01: drumul scade la **17%**, clusterizarea rămâne 97%
- Zona dintre p = 0,001 și 0,1: **lume mică**
- p = 1: rețea aleatoare, fără grupuri locale

Rețele fără scară: puține huburi, multe noduri cu puține legături



- 10 000 de noduri, grad mediu 6 în ambele rețele
- Grad maxim: **273** (BA) vs 16 (aleatoare)
- Cele mai conectate 1% dintre noduri au 11% din legături
- Coada BA scade ca o putere: $P(\text{grad} \geq k) \sim k^{-2}$

Epidemii pe rețele: aceeași boală, rețele diferite

Rețeaua (10 000 de noduri, grad mediu 6)	Infectați în total	Vârful (simultan)
Aleatoare (Erdős–Rényi)	55,5%	7,0%
Fără scară (Barabási–Albert)	55,3%	14,3%
BA, 10% vaccinați la întâmplare	43,5%	10,4%
BA, 10% vaccinați: cei mai conectați	0,3%	0,1%

- Transmisie 0,06 pe legătură și zi, vindecare 0,2 pe zi; medie pe 10 rulări (doar epidemiile care pornesc)
- Pe rețeaua fără scară, huburile se infectează devreme: vârful este de **două ori** mai înalt
- Aceleași 1 000 de doze: la întâmplare reduc epidemia cu un sfert; date hubu rilor, o **opresc**

Ce înseamnă rețelele pentru politicile de sănătate

- **Superrăspânditorii**: o mică parte dintre infectați produc majoritatea transmiterilor – efectul huburilor
- **Vaccinarea prin cunoștințe** (Cohen et al., 2003): alegem oameni la întâmplare și vaccinăm un prieten al fiecăruia – prietenii sunt în medie mai conectați
- **Măsuri țintite**: închiderea marilor evenimente reduce mai mult decât reducerea uniformă a contactelor
- **Urmărirea contactelor** funcționează mai bine înainte ca boala să ajungă la huburi
- Toate aceste concluzii **nu** apar într-un model cu ecuații pentru o populație omogenă

P **Paradoxul prieteniei**

Feld (1991): în medie, prietenii tăi au mai mulți prieteni decât tine – pentru că oamenii populari apar pe mai multe liste de prieteni.

Verificare

? Avem doze pentru **10%** dintr-o populație cu rețea de contacte fără scară. Ce strategie oprește cel mai bine epidemia?

A vaccinarea la întâmplare

B vaccinarea celor mai conectate persoane (huburile)

C vaccinarea celor cu cele mai puține contacte

D nu contează, doar procentul contează

Modelele orei 2 dintr-o privire

Model	Agentul	Regula esențială	Fenomenul emergent
Boids	reactiv, spațiu continuu	separare, aliniere, coeziune	stoluri ordonate ($\phi \approx 1$)
Schelling	reactiv, grilă	mută-te dacă vecinii similari < t	cartiere separate
Deneubourg	cu stare (memorie)	ridică izolat, lasă lângă altele	grămezi de cadavre
Chemotaxie	cu stare (o valoare)	rostogolire rară dacă crește	deplasare spre hrană
SIR pe rețea	cu stare (S, I, R)	infectare pe legături	vârfuri, rolul huburilor

Rezumatul orei 2

- ✓ **Boids**: separare, aliniere, coeziune; parametrul de ordine crește de la 0,08 la 0,99; fără aliniere nu apare ordinea
- ✓ **Schelling** (demo): prag 30% produce 74% vecini similari; peste ~0,8 nu mai există echilibru
- ✓ **Furnicile** Deneubourg: grămezi din fluctuații statistice; stigmergie
- ✓ **Chemotaxia**: o singură valoare memorată ajunge pentru a găsi hrana
- ✓ **Rețele**: lume mică, huburi; vaccinarea huburilor oprește epidemia

Ora 3

Ingineria unui model bazat pe agenți și frontiera

1. Etapele construirii unui ABM (Holban)
2. Protocolul ODD: cum descriem un model
3. Replicări, sensibilitate, calibrare ABC, validare
4. Unelte, scară și agenți generativi cu LLM

Etapele construirii unui model bazat pe agenți

1

Scopul

Ce întrebare? De ce agenți și nu ecuații?

2

Agenții

Ce entități, ce atribute statice și dinamice?

3

Comportamentul

Reguli, decizii, memorie, învățare

4

Mediul și interacțiunile

Grilă, spațiu, rețea; cine cu cine

5

Experimentele

Ce măsurăm, câte replicări, ce scenarii

6

Validarea

Comparăm cu datele și cu tiparele observate

Întrebări înainte de a scrie cod (Holban, cap. 7)

Tema	Întrebări
Scopul	Ce sistem modelăm? La ce întrebări răspunde modelul? Ce aduce în plus abordarea cu agenți?
Agenții	Ce entități sunt agenți? Ce relații modelăm prin ei? Ce date descriu un agent?
Structura	Ce atribute statice și dinamice are agentul? Cine le actualizează? Se poate deplasa?
Comportamentul	Ce decizii ia și pe baza căror informații? Reguli dacă–atunci, probabilități, regresii, rețele neuronale?
Interacțiunile	Cum interacționează cu alți agenți și cu mediul? Cât de generale sunt interacțiunile?
Rezultatele	Ce experimente, ce date urmărim? Cum validăm comportamentul și mecanismele de interacțiune?

Probleme pe care orice ABM trebuie să le rezolve

- **Conflictele**: doi agenți vor aceeași resursă în același moment – cine câștigă? (ordine aleatoare, prioritate, licitație)
- **Mediul pe straturi**: hartă, rețea socială, locuri de muncă – cum se deplasează agentul între ele
- **Reguli care se schimbă**: separăm clar activarea regulilor de mecanismul care creează reguli noi
- **Învățarea**: agentul își schimbă comportamentul din experiență (El Farol) – opțională, dar costisitoare
- Holban, după North și Macal (2011): aceste aspecte trebuie decise explicit, nu lăsate în seama codului

K **KISS sau KIDS?**

Axelrod: „Keep It Simple, Stupid” – Edmonds și Moss: „Keep It Descriptive, Stupid”. Pornim simplu și adăugăm detalii doar când datele sau întrebarea o cer.

De ce construim un model bazat pe agenți? (Epstein, 2008)

- **Explicație**: arătăm că un mecanism simplu este suficient pentru un fenomen (Schelling, furnicile)
- **Scenarii**: comparăm politici („ce-ar fi dacă…”) – vaccinare țintită, rute noi, reguli de piață
- **Ghidarea colectării de date**: modelul arată ce mărimi trebuie măsurate (analiza de sensibilitate)
- **Predicție**: posibilă doar după calibrare și validare atentă – și adesea doar pe termen scurt
- **Comunicare și predare**: un model vizual face un mecanism intuitiv (NetLogo, Mesa în browser)

E **Epstein**

„Why model?” (JASSS, 2008): există cel puțin 16 motive pentru a modela în afară de predicție. Scopul declarat decide cât de riguroasă trebuie să fie validarea.

Grimm et al., 2006, 2010, 2020

Protocolul ODD

1. Overview, Design concepts, Details
2. Cele 7 elemente
3. Exemplu completat: Schelling
4. Conceptele de proiectare

Protocolul ODD

Overview – Design concepts – Details

Un format standard pentru descrierea modelelor bazate pe agenți: aceleași secțiuni, în aceeași ordine, pentru orice model. Cititorul găsește imediat ce face modelul și poate să îl **reimplementeze** fără codul original.

Grimm et al., Ecological Modelling 198, 2006; JASSS 23(2), 2020

- **Overview**: scop și tipare, entități și variabile, procese și ordinea lor
- **Design concepts**: emergență, adaptare, interacțiune, stocasticitate, observare…
- **Details**: inițializare, date de intrare, submodele (ecuații, parametri)
- Folosit în mii de articole; cerut de multe reviste de ecologie și științe sociale

Cele 7 elemente ale protocolului ODD

	Element	Întrebarea la care răspunde
1	Scop și tipare	Ce vrea să explice modelul și după ce tipare îl judecăm?
2	Entități, variabile, scări	Ce agenți și ce mediu? Ce variabile au? Ce unitate de timp și spațiu?
3	Procese și ordonare	Ce se întâmplă la fiecare pas și în ce ordine? Sincron sau asincron?
4	Concepte de proiectare	Ce emerge? Cum percep, decid, învață agenții? Unde intervine aleatorul?
5	Inițializare	Care este starea la t = 0? Este aceeași la fiecare rulare?
6	Date de intrare	Ce date externe variază în timp (vreme, politici, prețuri)?
7	Submodele	Detaliile fiecărui proces: ecuații, reguli, valori ale parametrilor

Exemplu: modelul Schelling descris cu ODD (rezumat)

Element	Modelul nostru
Scop și tipare	Pot preferințe slabe produce segregare puternică? Tipar: fracțiunea de vecini similari
Entități și scări	Locuitori (grup 1 sau 2); grilă 50 × 50, toroidală; un pas = o rundă de mutări
Procese	Fiecare locuitor, în ordine aleatoare: verifică vecinii, se mută dacă e nemulțumit
Concepte	Emergență: cartierele; percepție: 8 vecini; stocasticitate: ordinea, casa nouă
Inițializare	90% ocupare, grupuri egale, poziții aleatoare (seed raportat)
Date de intrare	Niciuna (model abstract)
Submodele	Mulțumit dacă vecini similari / vecini ocupați ≥ t; t ∈ [0,1; 0,8]

Concepte de proiectare (Design concepts)

1 Emergență

Ce rezultate apar din interacțiuni și ce rezultate sunt impuse direct prin reguli?

2 Adaptare și obiective

Ce încearcă agenții să atingă? Cum își schimbă comportamentul?

3 Percepție

Ce variabile cunosc agenții, din ce rază, cu ce erori?

4 Interacțiune

Directă sau prin mediu? Locală sau globală?

5 Stocasticitate

Unde intervine aleatorul și de ce? Ce ar fi determinist?

6 Observare

Ce date colectăm din model pentru a-l testa și analiza?

Verificare

? Care este scopul principal al protocolului **ODD**?

A să accelereze simularea

B să descrie un model complet și standardizat, astfel încât alții să-l poată reimplementa

C să calibreze automat parametrii

D să înlocuiască validarea

Experimente

Replicări, sensibilitate, calibrare, validare

1. Câte rulări ne trebuie?
2. Analiza de sensibilitate
3. Calibrare bayesiană aproximativă (ABC)
4. Validare pe tipare

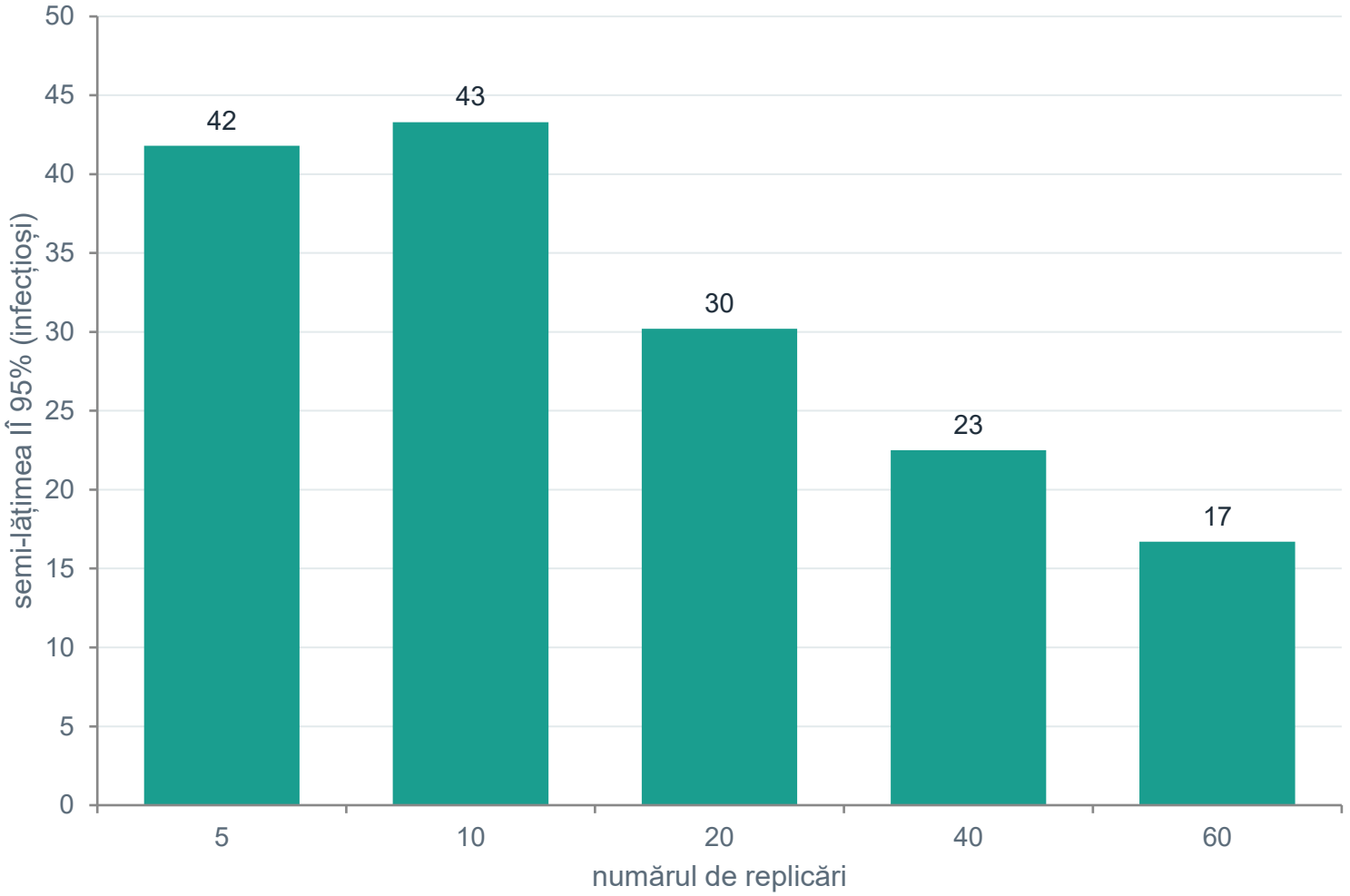
Stocasticitatea: o singură rulare nu înseamnă nimic

- Aleatorul apare peste tot: poziții inițiale, ordinea de activare, decizii probabiliste
- Raportăm **media** pe multe replicări și un **interval de încredere** (S6)
- Seed-uri controlate: numpy.random.SeedSequence.spawn – fluxuri independente, reproductibile
- **Numere aleatoare comune** (CRN) când comparăm scenarii: aceleași seed-uri pentru A și B (S6)
- Distribuția rezultatelor contează, nu doar media: probabilitatea de stingere, cazurile extreme

! **Larooij și Törnberg**

Una dintre criticile aduse simulărilor cu agenți generativi: multe studii raportează **o singură rulare**, deși modelul este puternic stocastic.

Câte replicări? Vârful epidemiei în modelul Holban



- 60 de rulări, 2 500 de agenți, fără imunitate care expiră
- Vârful: media **1 164**, abaterea 66 (CV 5,7%)
- Între 950 și 1 283 – o rulare poate greși cu peste 200
- Intervalul se îngustează ca 1/√n; ~0,2 s pe rulare

Numărul de replicări pentru o precizie dorită

$$n \approx (1{,}96 \cdot s / h)^2$$

s	abaterea standard estimată din câteva rulări pilot (aici s ≈ 66 infecțioși)
h	semi-lățimea dorită a intervalului de încredere de 95% (precizia)
exemplu	pentru h = 20: n ≈ (1,96 · 66,1 / 20)² ≈ **42** de replicări

Aceeași formulă ca în S6, R = (t·s/h)². Pentru ABM costisitoare, numărul de replicări se înmulțește cu numărul de scenarii și de valori ale parametrilor – de aici nevoia de paralelism (S8) și de modele surogat (S10).

Experimente în Mesa: colectarea datelor și baleierea parametrilor

```
class SchellingDate(Schelling):
    def __init__(self, **kw):
        super().__init__(**kw)
        self.datacollector = mesa.DataCollector(
            model_reporters={"segregare": segregare})
    def step(self):
        super().step()
        self.datacollector.collect(self)

rez = mesa.batch_run(SchellingDate,
    parameters={"prag": [0.2, 0.3, 0.5, 0.7],
                "seed": range(20)},      # 20 de replicări
    max_steps=100, data_collection_period=-1,
    number_processes=None)                # toate nucleele
df = pd.DataFrame(rez).groupby("prag")["segregare"]
print(df.mean(), df.std())
```

- DataCollector: ce măsurăm la fiecare pas (ODD: observare)
- batch_run: toate combinațiile de parametri × replicări
- number_processes=None: rulări în paralel pe toate nucleele
- Rezultatul: un tabel pandas, gata de analiză

Analiza de sensibilitate: ce parametri contează?

- **Unul câte unul** (OAT): variem un parametru, ceilalți fixați – simplu, dar ratează interacțiunile
- **Baleiere completă**: grilă de valori (ca pragul Schelling); costul crește exponențial cu numărul de parametri
- **Planuri factoriale 2^k** (S6) și eșantionare Latin Hypercube pentru multe dimensiuni
- **Indici Sobol**: ce fracțiune din variația rezultatului se datorează fiecărui parametru și interacțiunilor
- Atenție la **tranziții**: Schelling peste 0,8, El Farol cu predictori identici – comportamentul se schimbă brusc

T **Tornado (S6)**

Aceleași grafice tornado din săptămâna a șasea arată, pentru un ABM, ce parametri trebuie măsurați cel mai precis.

Calibrarea fără verosimilitate: ABC

Approximate Bayesian Computation (calcul bayesian aproximativ)

Pentru un ABM nu putem scrie probabilitatea datelor în funcție de parametri, dar putem **simula**. Ideea ABC: alegem parametri dintr-o distribuție a priori, simulăm, păstrăm parametrii ale căror simulări seamănă cel mai mult cu datele.

Beaumont, Zhang și Balding, Genetics, 2002; legătura cu calibrarea din S7

- 1. Eșantionăm θ din distribuția a priori (aici p ~ uniform pe [0,02; 0,2])
- 2. Simulăm modelul cu θ și calculăm distanța dintre simulare și date
- 3. Păstrăm cele mai apropiate 10% dintre simulări
- Parametrii păstrați aproximează distribuția a **posteriori**: valoare plus incertitudine

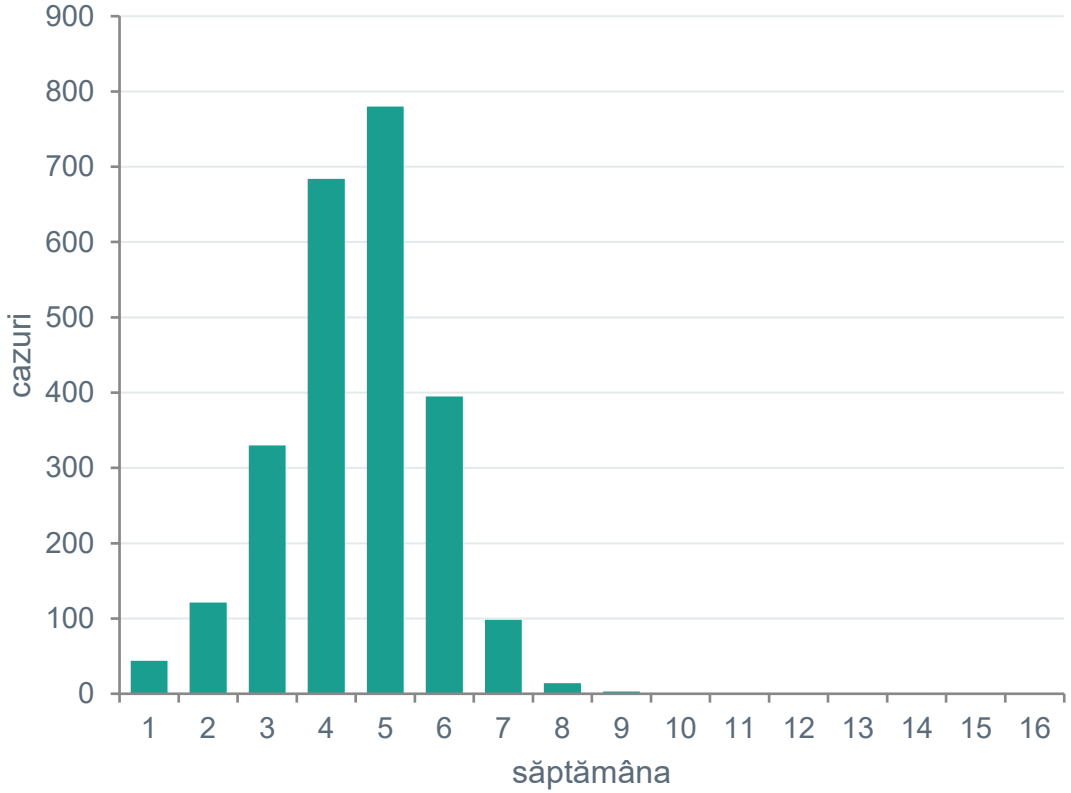
ABC prin respingere pentru modelul Holban

```
obs = saptamanal(cazuri_observate)          # 16 săptămâni
prior = rng.uniform(0.02, 0.2, 400)
dist = []
for k, p in enumerate(prior):
    sim = model_holban(p=p, seed=5000 + k)
    d = np.sqrt(np.mean((saptamanal(sim) - obs)**2))
    dist.append(d)
dist = np.array(dist)
prag = np.quantile(dist, 0.10)               # cele mai bune 10%
acceptat = prior[dist <= prag]
print(acceptat.mean(), np.quantile(acceptat, [0.05, 0.95]))
```

- „Datele observate”: o rulare cu p = 0,1, ascunsă algoritmului
- Distanța: RMSE între cazurile săptămânale
- 400 de simulări (2 500 de agenți): ~82 s
- Variante mai eficiente: ABC-SMC, emulatoare (S10)

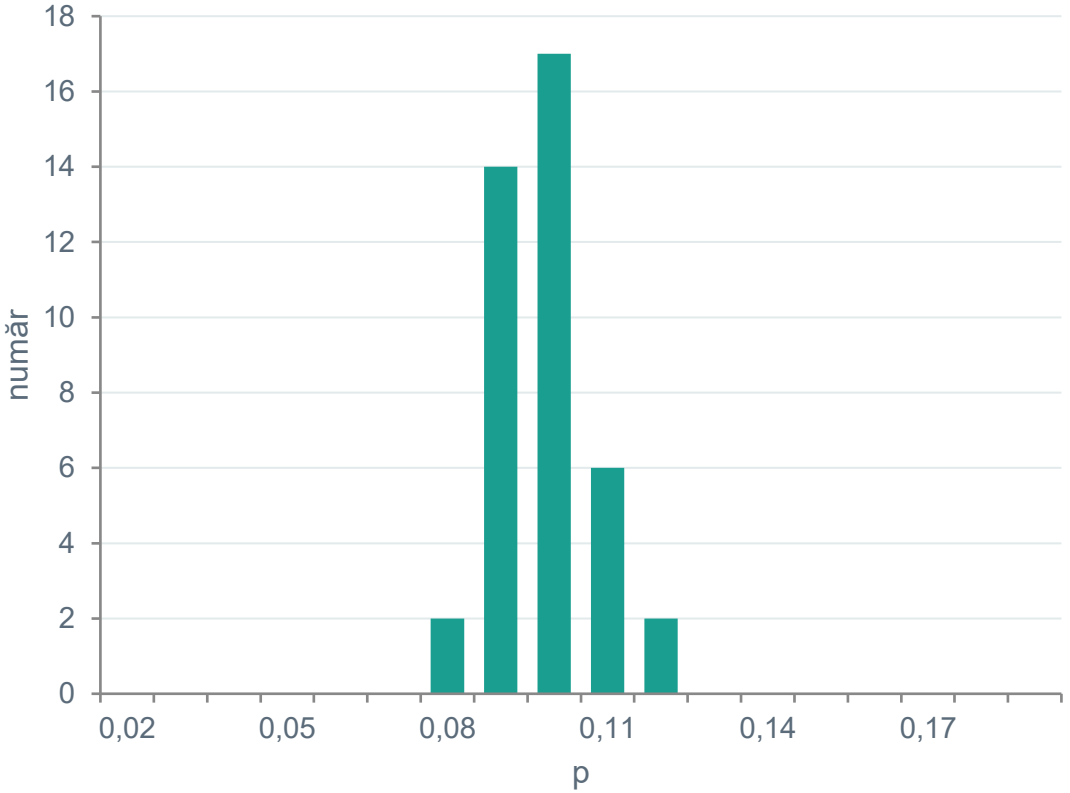
Datele și distribuția a posteriori a probabilității de transmisie

cazuri noi pe săptămână („observate”)



o singură epidemie, 2 500 de agenți

cele 41 de valori acceptate



p adevărat = 0,1; media a posteriori 0,102

Rezultatul calibrării ABC

0,102

media a posteriori a lui p (adevărat: 0,1)

[0,091; 0,116]

interval **credibil de 90%** – include valoarea adevărată

41 / 400

simulări **acceptate** (cele mai apropiate 10%)

- Distribuția a priori acoperea [0,02; 0,2]; datele au restrâns-o de aproximativ 7 ori
- Cu date reale apar și erori de model: ABC găsește cei mai buni parametri ai unui model care poate fi greșit – de aici validarea

Validarea unui model bazat pe agenți

- **Validare de față**: experții recunosc comportamentul ca plauzibil – necesară, dar slabă
- **Modelare orientată pe tipare** (Grimm et al., 2005): modelul trebuie să reproducă **simultan** mai multe tipare observate
- Exemple de tipare: indicele de segregare, numărul de grămezi, viteza frontului epidemic, distribuția gradelor
- **Docking**: două implementări independente (de ex. NetLogo și Mesa) trebuie să dea aceleași rezultate
- Validare pe **date nefolosite** la calibrare – ca în S6 și S7

V **Sargent (S6)**

Ciclul verificare–validare din săptămâna a șasea rămâne valabil; pentru ABM, validarea mecanismelor (cum decid agenții) este la fel de importantă ca validarea rezultatelor.

Verificare

? De ce folosim **ABC** pentru calibrarea unui ABM, în loc de metoda celor mai mici pătrate cu un optimizator clasic?

A pentru că ABC este întotdeauna mai rapid

B pentru că ABM nu are o funcție de verosimilitate explicită, iar rezultatele sunt aleatoare; ABC are nevoie doar de simulări

C pentru că ABC nu are nevoie de date

D pentru că modelele cu agenți nu au parametri

Practică

Unelte și modele la scară mare

1. NetLogo, Mesa, AnyLogic și altele
2. Performanța unui ABM
3. Orașe, țări, pandemii

Unelte pentru modelarea bazată pe agenți

Unealta	Caracteristici	Potrivită pentru
NetLogo 7	limbaj propriu, simplu; bibliotecă mare de modele; versiunea 7 din septembrie 2025	predare, prototipuri rapide
Mesa 3	Python; integrare cu NumPy, pandas; vizualizare în browser (3.5, martie 2026)	cercetare, analiză de date, acest curs
AnyLogic	comercial; agenți + evenimente discrete + dinamica sistemelor	industrie, logistică (folosit de Holban)
Repast4Py, MASON	Python cu MPI, respectiv Java performant	modele mari, calcul paralel
GAMA, MATSim	hărți reale (GIS); transport la scara unei țări	orașe, trafic, evacuări
AgentTorch	tensori pe GPU, diferențiabil, agenți bazați pe LLM	milioane de agenți, calibrare cu gradient

Performanța unui model bazat pe agenți

- Căutarea vecinilor: naiv $O(N^2)$; cu grilă sau arbore k-d mult mai puțin (Cernazean: construcție $O(N \log N)$) – ca la particule (S8)
- Vectorizare: un pas Boids cu N = 300 este de ~16 de ori mai rapid în NumPy decât cu bucle Python
- Agenți ca **tabele** (o coloană per atribut) în loc de obiecte: modelul Holban, 10 000 de agenți × 365 de zile în sub o secundă
- Paralelism: replicările și scenariile sunt „jenant de paralele”; un singur model mare cere descompunerea domeniului (S8)
- Actualizarea asincronă limitează paralelismul în interiorul unei rulări

P **Regula practică**

Mai întâi un model corect și lizibil (obiecte, Mesa); vectorizăm sau trecem pe GPU doar când știm exact ce vrem să rulăm de mii de ori.

Modele bazate pe agenți la scară mare

1 Transport

MATSim: fiecare locuitor cu programul zilnic și ruta lui, pentru orașe și țări întregi.

2 Pandemii

CovidSim (Imperial College, Raportul 9, S7), Covasim: agenți în case, școli, locuri de muncă.

3 Orașe cu LLM

AgentTorch: 8,4 milioane de agenți pentru New York – COVID-19 și piața muncii.

4 Evacuări

Stadioane, aeroporturi, clădiri: fiecare pieton cu viteză și rută proprie.

5 Piețe

Piețe financiare artificiale (Cernazean: bursa din New York), piețe de energie.

6 Gemeni digitali

Modele de oraș actualizate continuu cu date reale – tema săptămânii a zecea.

Frontiera

Agenți generativi cu modele de limbaj

1. Smallville: 25 de agenți (Park et al., 2023)
2. Simulări ale unor oameni reali (2024)
3. Milioane de agenți prin arhetipuri
4. Problema centrală: validarea

Agenți generativi (Park et al., 2023)

25 de agenți într-un oraș virtual, „Smallville”

Fiecare agent are o descriere în limbaj natural (profesie, familie, relații). Un model de limbaj generează ce face agentul, ce spune și ce plănuiește, pe baza unui **flux de memorie** cu tot ce a observat.

J. S. Park et al., Generative agents: interactive simulacra of human behavior, UIST 2023

- **Memorie**: toate observațiile, cu marcaj de timp; se extrag cele recente, importante și relevante
- **Reflecție**: periodic, agentul trage concluzii de nivel înalt din amintiri
- **Planificare**: planuri zilnice, revizuite când apar evenimente
- Prezentat la UIST 2023; agenții au fost evaluați de oameni ca **credibili**

Smallville: petrecerea de Ziua Îndrăgostiților

52%

agenți care știau de petrecere după 2 zile de joc (13 din 25), pornind de la **unul** singur (4%)

5 din 12

invitați care au ajuns efectiv la petrecere, la ora potrivită

0,74

densitatea rețelei de cunoștințe dintre agenți, față de 0,17 la început

- Pornind de la o singură idee dată unui agent („vrea să organizeze o petrecere”), ceilalți au răspândit invitațiile, s-au invitat la întâlniri și s-au coordonat – fără reguli scrise pentru aceste comportamente
- Dintre cei 7 care nu au venit: 3 au avut alte planuri, 4 au spus că vin, dar nu și-au planificat drumul

Simularea unor oameni reali (Park et al., 2024)

- **1 052** de persoane reale, intervievate câte două ore; fiecare devine un agent generativ
- Agenții răspund la General Social Survey **85%** la fel de precis cum își reproduc oamenii propriile răspunsuri după două săptămâni
- Rezultate comparabile pentru trăsăturile de personalitate și pentru replicarea unor experimente
- Promisiune: „laboratoare” sociale în care testăm politici înainte de a le aplica
- Riscuri: confidențialitate, folosirea abuzivă a unor copii digitale ale oamenilor

? Cum citim cifra de 85%

Este o precizie **relativă**: oamenii nu își repetă perfect răspunsurile; agenții ating 85% din această consecvență umană, nu 85% răspunsuri corecte.

Milioane de agenți: arhetipuri în loc de apeluri individuale

- Un apel la un model de limbaj pentru fiecare agent și fiecare pas este **prea scump** pentru milioane de agenți
- AgentTorch (MIT, AAMAS 2025): agenții cu aceleași caracteristici formează un **arhetip**; modelul de limbaj este întrebat o dată per arhetip
- **8,4 milioane** de agenți pentru New York: răspândirea COVID-19 și participarea pe piața muncii
- Simulare pe GPU, diferențiabilă: parametrii se pot calibra cu gradienți
- Compromis: expresivitatea agenților generativi vs costul și numărul de agenți

G **Legătura cu S8**

Aceleași idei de performanță: vectorizare, GPU, evitarea calculelor repetate – aplicate unor agenți care „gândesc” în limbaj natural.

Agenți generativi: promisiuni și probleme

Promisiuni

- comportament **realist**, fără reguli scrise de mână
- agenții vorbesc, negociază, își schimbă opinia
- eterogenitate bogată, din descrieri în limbaj natural
- scenarii noi, testate rapid

vs

Probleme (Larooij și Törnberg)

- **cutie neagră**: nu știm de ce agentul face ce face
- **stereotipuri** și prejudecăți culturale ale modelului
- cost mare, rezultate greu de reprodus
- validare mai ales „de față”, adesea o singură rulare

Analiza a 35 de studii (Artificial Intelligence Review, 2026): modelele de limbaj pot **agrava**, nu rezolva, problema validării ABM – nu au nici simplitatea modelelor formale, nici fundamentarea empirică a celor bazate pe date.

Cum am valida un model cu agenți generativi

- Comparăm cu **date empirice** la nivel individual și agregat, nu doar cu impresia de credibilitate
- Rulăm **multe replicări** și raportăm variabilitatea – ieșirile modelelor de limbaj sunt stocastice
- **Ablații**: înlocuim agentul generativ cu reguli simple – ce câștigăm de fapt?
- **Docking** cu modele clasice: reproduce agentul generativ Schelling sau El Farol în condiții controlate?
- Transparență: ODD extins cu prompturile, versiunea modelului de limbaj, temperatura

V **Aceleași principii**

Agenții generativi nu schimbă regulile jocului din S6: verificare, validare pe date, replicări, incertitudine raportată.

Verificare

? Un studiu cu agenți generativi raportează că agenții „par foarte realiști” pe baza evaluării unor oameni, dintr-o singură rulare. Care este principala problemă?

A modelul de limbaj este prea mic

B validarea este doar „de față” și ignoră stocasticitatea; lipsește comparația cu date empirice

C agenții sunt prea puțini

D nu s-a folosit protocolul ODD pentru cod

Responsabilitatea modelatorului

- Modelele cu agenți au influențat decizii reale: carantine, vaccinare, planificarea transportului
- Comunicăm **incertitudinea**: intervale, scenarii, ipoteze – nu o singură curbă
- **Reproductibilitate**: cod public (de ex. CoMSES Net), ODD, seed-uri, versiuni ale bibliotecilor
- Datele individuale cer **confidențialitate**; agenții generativi pot reproduce **prejudecăți**
- Box: „Toate modelele sunt greșite, dar unele sunt utile” – spunem clar pentru ce este util modelul nostru

C **Lecția din 2020 (S7)**

Codul CovidSim a fost verificat independent (CODECHECK) după critici publice: transparența a crescut încrederea în rezultate.

Rezumatul săptămânii

- ✓ Agentul: percepție, stare, comportament, acțiune; mediu, topologie, actualizare **asincronă** cu ordine amestecată
- ✓ Epidemia Holban: spațiul schimbă vârful și produce valuri; El Farol: diversitatea strategiilor contează
- ✓ Modele clasice: **Boids**, **Schelling** (30% produce ~74%), furnicile Deneubourg, chemotaxia, rețele și huburi
- ✓ **ODD** pentru descriere; replicări $n \approx (1{,}96 \cdot s/h)^2$; calibrare **ABC**; validare pe tipare
- ✓ Agenți generativi: promițători, dar **validarea** rămâne problema centrală

Săptămâna viitoare

Modele din date, AI + simulare și gemeni digitali

- De la regresie la modele surogat care învață o simulare scumpă
- Fizică + învățare automată; simulare pentru antrenarea AI
- Gemeni digitali și sinteza cursului

Calibrarea ABC a cerut 400 de simulări. Ce facem când o singură simulare durează o oră? Putem învăța o rețea neurală să o imite?

Bibliografie pentru săptămâna 9

- Ș. Holban – Tehnici de modelare și simulare, cap. 7: modelarea bazată pe agenți (materiale de curs); Prof. Cernazean – Modelling and Simulation, Lecture 8: Agent based models
- T. C. Schelling – Dynamic models of segregation, Journal of Mathematical Sociology, 1971; Micromotives and Macrobehavior, 1978
- C. Reynolds – Flocks, herds and schools: a distributed behavioral model, SIGGRAPH 1987
- J.-L. Deneubourg et al. – The dynamics of collective sorting, 1991; W. B. Arthur – Inductive reasoning and bounded rationality, AER, 1994
- V. Grimm et al. – The ODD protocol for describing agent-based and other simulation models: a second update, JASSS 23(2), 2020
- S. F. Railsback, V. Grimm – Agent-Based and Individual-Based Modeling: A Practical Introduction, ed. 2, Princeton, 2019
- J. S. Park et al. – Generative agents, UIST 2023; Generative agent simulations of 1,000 people, 2024
- M. Larooij, P. Törnberg – Validation is the central challenge for generative social simulation, Artificial Intelligence Review, 2026
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5